06 · Server-Verfahren

Ereignisarbeit budgetieren und zeitlich einplanen · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 50

06 · Ereignisse mit Rechenzeit versorgen

- Server-Verfahren für aperiodische Arbeit
- Background, Polling, Deferrable und Sporadic Server
- Budgets lesen, Zeitachsen zeichnen, Fristen prüfen
- Papieraufgaben und eigene FreeRTOS-Modellsimulation

Notizen 1 / 50

Ein Bedienkommando oder Diagnoseauftrag trifft nicht unbedingt im Takt eines Reglers ein. Trotzdem teilt es sich mit diesem denselben Prozessor. Ein Server-Verfahren ordnet dieser ereignisgetriebenen Arbeit eine geregelte Bedienung zu. Server bezeichnet hier einen Mechanismus zur Zuteilung von Rechenzeit innerhalb eines Echtzeitsystems, keinen Rechner im Netzwerk. Aperiodisch bedeutet in diesem Termin, dass kein kleinster Ankunftsabstand zugesichert ist.
Background heißt Hintergrundbearbeitung, Polling regelmäßiges Nachsehen, Deferrable aufschiebbar und Sporadic sporadisch. Wir entwickeln die Unterschiede an denselben Aufträgen, statt vier isolierte Namen auswendig zu lernen. FreeRTOS ist das im Kurs verwendete Echtzeitbetriebssystem; es liefert die feste Prioritätsauswahl, aber nicht automatisch diese Server-Verfahren. Die Beispiele sind ausdrücklich Modellrechnungen. Der praktische Teil macht eine Polling-Regel sichtbar; eine vollständige Implementierung des Sporadic Servers ist nicht Gegenstand dieses kurzen Versuchs.

Folie 2 / 50

Eine größere Queue schützt den Regler?

Ein Kern, feste Prioritäten:

- Ein Regler muss regelmäßig rechtzeitig rechnen.
- Eine höher priorisierte Diagnose-Task bekommt ständig neue Aufträge.
- Beide benötigen denselben Prozessor.

Reicht eine größere Queue, damit der Regler seine Frist einhält?
30 Sekunden nachdenken, dann eine Minute zu zweit begründen.

Notizen 2 / 50

Die Frage knüpft an Prioritäten aus Vorlesung 3 und Warteschlangen aus Vorlesung 5 an. Ein Regler ist hier die regelmäßige Berechnung einer Stellgröße mit einer geforderten Abschlussfrist. Die Diagnose-Task ist ein weiterer Thread, also ein fortsetzbarer Ausführungsablauf. Es gibt genau einen Kern mit festen Prioritäten. Für den Gedankenversuch nehmen wir an, dass immer neue Diagnoseaufträge bereitstehen und deren Bearbeitung ohne blockierende Pause fortgesetzt wird. Eine Queue ist der Speicher für wartende Aufträge.

Lassen Sie zunächst dreißig Sekunden einzeln überlegen, anschließend eine Minute zu zweit begründen und sammeln Sie kurz zwei unterschiedliche Vermutungen. Plausible Antworten sind „Ja, weil mehr Aufträge gespeichert werden können“ oder „Nein, die hohe Priorität bleibt“. Die erste Aussage verwechselt Speicherplatz mit Rechenzeit. Solange die höhere Task ausführungsbereit bleibt, bekommt der niedrigere Regler in diesem Modell keinen Prozessor. Mehr Queueplätze ändern daran nichts. Sie können eine Lastspitze aufnehmen, erzeugen aber weder zusätzliche Rechenkapazität noch eine Fristgarantie.

Eine niedrigere Diagnosepriorität könnte den Regler schützen, würde aber die Bedienung der Diagnose ebenfalls von verfügbarer Zeit abhängig machen. Heute entwickeln wir deshalb einen zusätzlichen Vertrag: Wie viel eigene Prozessorzeit darf Ereignisarbeit erhalten, wann steht diese Zeit bereit und welche anderen Fristen bleiben erfüllt? Diese Auflösung muss vor der Diskussion noch nicht bekannt sein. Die spätere Folie „Klausurcheck: sechs Angaben vor jeder Zeitachse“ nimmt die Frage mit dem erarbeiteten Budget- und Nachweisbegriff wieder auf.

Folie 3 / 50

Vorwissen wird zur neuen Frage

Aus Vorlesungen 1–5	Neue Verwendung
Rechenbedarf und Antwortzeit	Budgetverbrauch richtig zählen
Feste Prioritäten und Fristnachweis	Server neben Regler einplanen
Threads und Queues	Aufträge sammeln und bedienen
Durchsatz und Pufferkapazität	Lastspitze und Dauerüberlast unterscheiden

Notizen 3 / 50

Der Rechenbedarf eines Jobs zählt seine eigene Prozessorarbeit. Seine Antwortzeit beginnt dagegen mit der Freigabe und endet erst mit dem Abschluss; sie enthält Wartezeiten. Feste Prioritäten ordnen die bereiten Ausführungspfade dauerhaft. Ein Thread ist ein vom Betriebssystem verwalteter Ausführungspfad, eine Queue eine Warteschlange für Nachrichten oder Aufträge. Diese Begriffe werden nun zusammengeführt: Die Queue speichert ankommende Arbeit, ein Server entscheidet über ihre zeitliche Bedienung.

Vorlesung 5 hat gezeigt, dass eine Queue eine begrenzte Lastspitze puffern kann. Dauerhaft mehr eingehende als verarbeitbare Arbeit füllt dagegen jeden endlichen Puffer. Wir bleiben beim bekannten Modell eines Prozessorkerns mit festen Prioritäten und gegebenen eigenen Rechenbedarfen. Die neue Frage lautet, wann und wie viel Prozessorarbeit der Consumer für seine wartenden Aufträge erhalten darf. Dafür verbinden wir den Datenvertrag aus Vorlesung 5 mit der Antwortzeitanalyse aus Vorlesung 2. Zuerst legen wir die zugelassene Last und die Bedienregel fest; danach prüfen wir die daraus folgenden Fristen.

Folie 4 / 50

Fünf überprüfbare Lernziele

1. Auftrag, Serverbudget und Priorität auseinanderhalten
2. Vier Bedienverfahren an einer Zeitachse vergleichen
3. Beim Sporadic Server Rückgaben nachvollziehbar führen
4. Polling mit Antwortzeitanalyse einplanen
5. Fristen, Überlast und Implementierungsgrenzen begründen

Notizen 4 / 50

Ein Auftrag ist die einzelne eingegangene Arbeit. Das Serverbudget ist die noch erlaubte eigene Prozessorzeit des Bedienmechanismus; die Priorität bestimmt dessen Rang gegenüber anderen bereiten Aufgaben. Aus diesen drei Angaben entstehen noch keine identischen Antwortzeiten. Sie sollen die Entscheidung an jedem Ereigniszeitpunkt begründen können: Ist Arbeit vorhanden, ist Budget vorhanden und darf der Server gegenüber seinen Konkurrenten laufen?
Sporadic Server bezeichnet das Verfahren, das verbrauchte Kapazität nach Regeln wieder auffüllt, um die Einplanung mit periodischer Arbeit zu ermöglichen. Wir rechnen vollständig definierte Einstiegsfälle ohne höher priorisierte Störer. Polling bedeutet eine regelmäßige Bediengelegenheit; dafür benutzen wir die bekannte Antwortzeitanalyse. Ein Fristnachweis prüft den Abschluss bis zur geforderten Deadline. Überlast heißt, dass mehr Arbeit angeboten wird, als unter den zugesicherten Bedingungen verarbeitet werden kann. Eine kleine Simulation oder eine priorisierte FreeRTOS-Task ersetzt keinen dieser Nachweise.

Folie 5 / 50

180 Minuten: verstehen, rechnen, beobachten

Tätigkeit	Minuten
Einstiegsfrage und Austausch	2
Modell, Background und Polling	33
Deferrable und Sporadic Server	35
Fristnachweis, Überlast und Umsetzung	20
Papieraufgaben mit Besprechung (Auswahl)	45
Eigene Modellimplementierung und Prüfung	30
Quiz und gemeinsame Klärung	15
Gesamt	180

Notizen 5 / 50

Vier Schulstunden mit je 45 Minuten ergeben 180 Minuten. Nach zwei Minuten Einstiegsfrage und Austausch führen 33 Minuten in Modell, Background und Polling ein. Weitere 35 Minuten vergleichen

Notizen, Fortsetzung 5 / 50

Deferrable und Sporadic Server; 20 Minuten verbinden die Rechenregeln mit den Grenzen einer Implementierung. Der 45-Minuten-Block wird aus den vier Papierfolien und Aufgaben 1 bis 5 des Übungsblatts zusammengestellt. Sie üben darin Rechenschritte und Begründungen, die ohne Entwicklungsumgebung in einer Klausur möglich sind.
Der Boardversuch setzt das bereits eingerichtete Projekt und funktionierende Ausführung auf dem Cortex-M4 voraus. Cortex-M4 bezeichnet den dafür verwendeten Prozessorkern des STM32H755. Neue Werkzeuginstallation und vollständige Serverprogrammierung gehören in die Vorbereitung. Der 30-Minuten-Block liegt im geschätzten Bereich von 25 bis 35 Minuten der Übungsaufgabe. Die Studierenden implementieren auf der vorbereiteten Basis selbst einen kleinen Polling-Modellschritt und die beiden Threadschleifen. Die Musterlösung erscheint nur im Lösungsprofil des Übungsblatts. Die Lehrperson kann aus dem Aufgabenpool auswählen, falls die praktische Integration länger dauert. Ohne Board können Sie dessen Zustandsfolge auf Papier prognostizieren und prüfen. Das ist dann eine Modellprüfung, keine Hardwaremessung. Im Quiz erklären wir besonders die unterschiedlichen Auffüllregeln und die Verwechslung zwischen Prozessorzeit und verstrichener Zeit.

Folie 6 / 50

Der Regler bekommt unregelmäßige Nachbarn

- Regler: regelmäßig freigegeben, eigene harte Frist
- Bedienkommando: trifft bei einer Benutzereingabe ein
- Diagnose: kann mehrere Aufträge hintereinander liefern
- Ein Kern: alle benötigen dieselbe Prozessorzeit

Welche Ereignisarbeit darf den Regler wie lange verdrängen?

Notizen 6 / 50

Der Regler steht für eine periodische Aufgabe, etwa die wiederholte Berechnung einer Stellgröße. Hart bedeutet, dass die spezifizierte Frist unter allen zugelassenen Bedingungen eingehalten werden muss. Bedienkommandos und Diagnoseaufträge können ohne festen Takt eintreffen. Ihre fachliche Bedeutung und ihre eigenen Fristen müssen gesondert festgelegt werden; ereignisgetrieben ist nicht automatisch gleich weich echtzeitkritisch.
Eine hoch priorisierte Task für alle Diagnoseaufträge wirkt zunächst reaktionsschnell. Ohne begrenzten Bedarf könnte sie aber unter einer Auftragsflut den Regler beliebig lange verdrängen. Ganz niedrige Priorität schützt die regelmäßige Arbeit besser, kann die Diagnose dagegen lange warten lassen. Ein Server soll die verfügbare Bedienung bewusst gestalten. Dabei bleibt die eine zentrale Ressource dieselbe: Auf einem Kern kann immer nur ein Ausführungspfad gleichzeitig rechnen. Eine Queue bewahrt Aufträge auf, erzeugt aber keine zusätzliche Rechenkapazität.

Folie 7 / 50

Aperiodischer Auftrag und sporadische Task

Ankunftsmodell	Zusicherung
Periodisch	regelmäßiger Abstand
Sporadisch	mindestens ein festgelegter Abstand
Aperiodisch	kein zugesicherter Mindestabstand

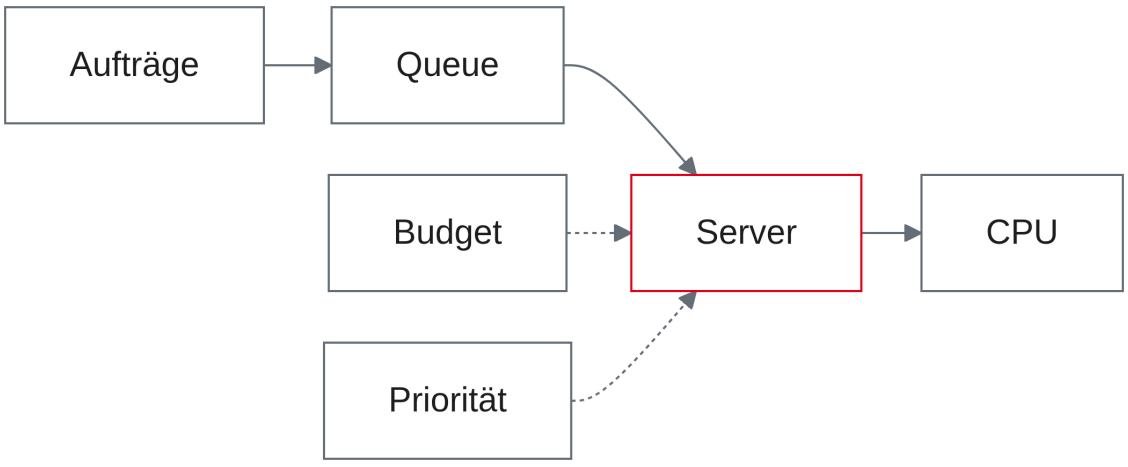
Der Name Sporadic Server legt den Eingabeabstand nicht fest.

Notizen 7 / 50

Eine periodische Task erzeugt Jobs in festen Abständen. Eine sporadische Task darf unregelmäßig freigegeben werden, hält aber einen zugesicherten Mindestabstand ein. Bei einem Mindestabstand von acht Millisekunden wären Freigaben bei eins und zehn zulässig, bei eins und vier dagegen nicht. Aperiodische Aufträge haben hier keine solche Zusicherung. Für eine harte Zusage brauchen auch sie ein begrenztes Lastmodell oder eine explizite Annahme- und Verwerfungsstrategie.
Sporadic Server ist der Eigenname eines Bedienverfahrens. Er bedeutet nicht, dass die eingehenden Bedienaufträge selbst plötzlich sporadische Tasks mit diesem Mindestabstand wären. Mehrere Aufträge dürfen gleichzeitig in seiner Queue liegen. Das Verfahren regelt die vom Server erzeugte Prozessorbelastung durch seine Kapazität und Wiederauffüllung. Diese Trennung ist für Klausurfragen wesentlich: Ein Parameter des Servers ist nicht automatisch ein Parameter jedes enthaltenen Auftrags.

Folie 8 / 50

Queue, Budget und Priorität



FIFO: ältester Auftrag zuerst.

Notizen 8 / 50

Eine Queue ist die Warteschlange der noch nicht abgeschlossenen Aufträge. FIFO bedeutet First In, First Out: Der zuerst eingegangene Auftrag wird zuerst bearbeitet. In unseren Beispielen wird ein unterbrochener Auftrag vor späteren Aufträgen fortgesetzt. Das Restbudget b misst die noch verfügbare eigene CPU-Zeit, also Prozessorzeit; CPU steht für Central Processing Unit. Die Serverpriorität ist sein fester Rang gegenüber anderen Aufgaben auf diesem Kern.
Eine volle Queue macht einen Server mit Budget null nicht ausführbar. Umgekehrt ist ein Server mit voller Kapazität und leerer Queue ebenfalls ohne Arbeit. Und selbst mit Arbeit und positivem Budget kann eine höher priorisierte Aufgabe zuerst laufen. Diese drei Fälle müssen getrennt sichtbar bleiben. Innerhalb der Queue könnten andere Regeln verwendet werden, etwa nach Auftragsfristen. Das würde die Reihenfolge einzelner Abschlüsse verändern und ist nicht Teil der heutigen FIFO-Rechnungen. Wir führen zunächst die Größe des Budgets ein, bevor wir seine Verfügbarkeit über die Zeit bestimmen.
Die Grafik wird entlang der durchgezogenen Pfeile von links nach rechts gelesen: Ein Ereignis erzeugt einen Auftrag, die Auftragsqueue bewahrt ihn auf und der Server führt seine Bearbeitung auf der CPU aus. Die gestrichelten Pfeile zeigen zwei zusätzliche Bedingungen, keine zeitlichen Schritte: Der Server benötigt verfügbares Budget und muss vom Scheduler anhand seiner festen Priorität ausgewählt werden. Auch eine nichtleere Queue allein erlaubt daher noch keine Ausführung. Budget und Scheduler sind hier logische Voraussetzungen, keine weiteren Prozessoren. Die Darstellung erklärt die Architektur auf einem Kern; sie legt weder feste Ausführungsfenster noch eine Wiederauffüllregel fest.

Folie 9 / 50

Kapazität und Zeitraum: Q und T

- Q: zugeordnete Kapazität in CPU-Millisekunden
- T: Zeitraum der jeweiligen Serverregel
- b: aktuell verfügbares Restbudget, $0 \leq b \leq Q$
- Beispiel: $Q = 2\,\text{ms}$, $T = 8\,\text{ms}$

$$U_S = \frac{Q}{T} = \frac{2}{8} = 25\,\%$$

Notizen 9 / 50

Q ist die Kapazität des Servers, gemessen in eigener CPU-Zeit; CPU heißt Central Processing Unit, Prozessor. T ist die zugeordnete Periode beziehungsweise der Wiederauffüllzeitraum der konkreten Regel. b bezeichnet das momentan verfügbare Restbudget. Die Kapazität ist eine Obergrenze für dieses Restbudget, kein unbegrenzt ansparbares Guthaben. Alle Zeitgrößen des Beispiels sind in Millisekunden angegeben.
Der Quotient $U_S = Q/T$ ist der nominell zugeordnete Prozessoranteil, hier zwei Achtel gleich 25 Prozent. Er bestimmt nicht, wann diese Zeit verfügbar wird. Polling, Deferrable und Sporadic Server können denselben Quotienten haben und trotzdem sehr unterschiedliche Auftragsantwortzeiten erzeugen. Auch die Behauptung, in jedem beliebig verschobenen Acht-Millisekunden-Fenster seien höchstens zwei Millisekunden Serverarbeit möglich, folgt nicht allgemein aus dem Quotienten. Für einen Fristnachweis benötigen wir die Auffüllregel, die Prioritäten und die konkurrierende Last. Background-Bearbeitung besitzt

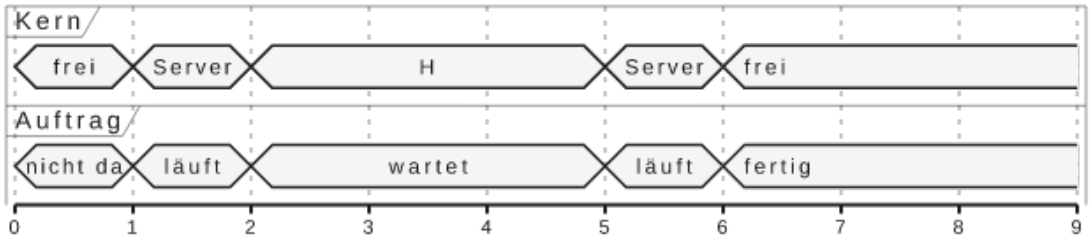
Notizen, Fortsetzung 9 / 50

dagegen kein solches reserviertes Budget.

Folie 10 / 50

Budget zählt eigene CPU-Zeit

Lehrmodell · Zeit in ms · fremde Arbeit ist keine Serverarbeit



$$C = (2-1) + (6-5) = 2\,\text{ms}, \qquad R = 6 - 1 = 5\,\text{ms}$$

Notizen 10 / 50

Die horizontale Achse des PlantUML-Timing-Diagramms zählt Millisekunden. Die obere Spur zeigt die Belegung des Kerns, die untere den Zustand des betrachteten Auftrags. H ist eine höher priorisierte andere Aufgabe. Der Auftrag kommt bei eins an, rechnet von eins bis zwei, wartet während H von zwei bis fünf und rechnet nochmals von fünf bis sechs. Die Wörter und Abschnittsgrenzen tragen die Bedeutung; die Linien sind keine gemessenen Signale.
CPU bedeutet Central Processing Unit, Prozessor. Der eigene Bedarf C ist die Summe beider eigenen Laufabschnitte, also eins plus eins gleich zwei Millisekunden. Die Antwortzeit R reicht von der Freigabe bei eins bis zum Abschluss bei sechs und beträgt fünf Millisekunden. Wenn das Budget nur eigene Serverarbeit belastet, verbraucht dieser Auftrag zwei Budgeteinheiten. Die fremde Arbeit wird nicht als zusätzliche eigene Serverzeit abgebucht. Dieses allgemeine Abrechnungsbild legt noch keine konkrete Wiederauffüllregel des Sporadic Servers fest. Dessen vollständige Behandlung bei höherpriorisierter Arbeit benötigt weitere Regeln.

Folie 11 / 50

Wann darf ein budgetierter Server laufen?

1. Mindestens ein Auftrag ist unbearbeitet.
2. Restbudget $b > 0$ ist verfügbar.
3. Keine höher priorisierte bereite Aufgabe verdrängt ihn.

Situation	Folge
Queue leer, Budget vorhanden	keine Serverarbeit
Queue gefüllt, Budget null	auf Auffüllung warten

Notizen 11 / 50

Ein budgetierter Server kann nur arbeiten, wenn sowohl eine fachliche Anforderung als auch die Erlaubnis zum Verbrauch von Rechenzeit vorhanden ist. Das Restbudget b ist diese noch verfügbare eigene Ausführungszeit. Die Queue ist die Warteschlange der unvollständigen Aufträge. Ein laufender Auftrag darf nach Budgeterschöpfung nicht heimlich weiterrechnen, nur weil sein Funktionsaufruf noch nicht fertig ist. Die Unterbrechung oder geeignete Zerlegung der Arbeit gehört deshalb zur Umsetzung des Verfahrens.
Die dritte Bedingung kommt vom Scheduler, der Auswahlregel des Echtzeitbetriebssystems: Auch ein berechtigter Server muss höher priorisierte bereite Arbeit vorlassen. Warten wegen leerer Queue und Warten wegen fehlenden Budgets sind unterschiedliche Ursachen. Welche Bedeutung eine leere Queue für das verbleibende Budget hat, ist gerade einer der Unterschiede zwischen Polling und den beiden anderen Verfahren. Für den ersten Vergleich lassen wir höher priorisierte Störer bewusst weg, damit allein die Budgetregel die unterschiedlichen Verläufe erklärt.

Folie 12 / 50

Ein gemeinsames, vollständig bestimmtes Modell

Ein Kern · keine Höherstörer · FIFO · Verwaltungskosten null.

Auftrag	Freigabe r	eigener Bedarf C
A	1	1
B	6	1
C	9	2

Alle Zeiten in ms · $Q = 2, T = 8$ · Anfangsbudget 2.

Notizen 12 / 50

Die folgenden Vergleichszeitachsen verwenden genau dieselben didaktisch vorgegebenen Aufträge. r

Notizen, Fortsetzung 12 / 50

ist die Freigabe beziehungsweise Ankunft, C der eigene Prozessorbedarf. A benötigt eine Millisekunde ab Freigabe eins, B eine ab sechs und C zwei ab neun. Die Buchstaben bezeichnen Aufträge, nicht verschiedene feste Prioritäten. FIFO bedeutet First In, First Out, Bedienung in Ankunftsreihenfolge. Es gibt keinen höher priorisierten Störer und keine Kosten für Umschalten, Queueverwaltung oder Budgetbuchhaltung.
$Q = 2$ ist die Kapazität, $T = 8$ der Serverzeitraum; die Einheit ist jeweils Millisekunden. Anfangs sind zwei Budgeteinheiten vorhanden. Bei Polling und Deferrable liegen die festen Auffüllgrenzen bei null, acht und sechzehn. An einer gemeinsamen Zeitgrenze werden zunächst fällige Auffüllungen und Ankünfte berücksichtigt, dann wird die nächste Ausführung ausgewählt. Ein Abschnitt von acht bis neun ist links geschlossen und rechts offen; sein Abschluss liegt bei neun. Auftragsfristen sind zunächst nicht gegeben. Wir vergleichen erst Antworten, bevor wir daraus Rechtzeitigkeit ableiten.

Folie 13 / 50

Background: Arbeit im Hintergrund

- Aperiodische Aufträge erhalten niedrigste Priorität.
- Sie laufen, wenn vorrangige Arbeit nicht bereit ist.
- Kein reserviertes Q/T, keine Auffüllliste.
- Viel freie Zeit hilft; dauernde Vordergrundlast kann sie verdrängen.

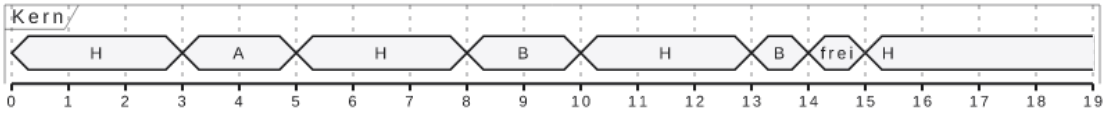
Notizen 13 / 50

Background Scheduling heißt Hintergrundbearbeitung. Die aperiodischen Aufträge ohne zugesicherten Mindestankunftsabstand werden hinter der zu schützenden periodischen Arbeit eingeordnet. Sind mehrere Hintergrundaufträge vorhanden, gilt hier First In, First Out, zuerst eingegangen zuerst bedient. Das Verfahren benötigt kein Kapazitätsbudget Q und keinen Wiederauffüllzeitraum T. Die Frage nach einem Verbrauchs- oder Rückgabetermin stellt sich daher nicht.
Unter dem idealen Einprozessormodell verdrängt reine Hintergrundrechnung keine höher priorisierte bereite Aufgabe. Reale gemeinsame Sperren oder Verwaltungsaufwand müssen trotzdem berücksichtigt werden. Für die Hintergrundaufträge selbst folgt keine allgemeine Antwortzeitschranke: Eine zulässige Vordergrundlast kann die verfügbare Restzeit klein machen oder vollständig aufbrauchen. Im gemeinsamen Modell ohne jeden Vordergrundstörer wären A, B und C sofort bei Ankunft bedienbar. Um die Abhängigkeit von tatsächlich freier Zeit zu sehen, betrachten wir deshalb auf der nächsten Folie ausdrücklich ein anderes Beispiel mit regelmäßiger Vordergrundarbeit.

Folie 14 / 50

Background nutzt Lücken der Vordergrundarbeit

Neues Lehrbeispiel · Vordergrund H: $C_H = 3$, $T_H = 5$ ms



A: $r = 1$, $C = 2$, Ende 5 · B: $r = 4$, $C = 3$, Ende 14.

Notizen 14 / 50

Die horizontale Achse des PlantUML-Diagramms zählt Millisekunden, die einzige Spur zeigt die Prozessorbelegung. H ist eine periodische Vordergrundaufgabe mit eigenem Bedarf $C_H = 3$ und Periode $T_H = 5$. Sie läuft von null bis drei, fünf bis acht, zehn bis dreizehn und fünfzehn bis achtzehn. A und B sind niedriger priorisierte Hintergrundaufträge; r bedeutet Freigabe und C eigenen Bedarf. Die freien Abschnitte sind für den Hintergrund nutzbar, aber keine reservierten Budgets.
A kommt bei eins mit Bedarf zwei an und rechnet von drei bis fünf. Sein Abschluss liegt bei fünf, die Antwortzeit beträgt fünf minus eins gleich vier. B kommt bei vier mit Bedarf drei an. Es wartet zunächst auf A und H, rechnet von acht bis zehn zwei Einheiten und von dreizehn bis vierzehn die dritte. Seine Antwortzeit beträgt vierzehn minus vier gleich zehn Millisekunden. Die Summe seiner eigenen Arbeit bleibt drei. Dieser eine Verlauf erklärt die Regel, beweist aber keine allgemeine maximale Wartezeit für beliebige Vordergrundlast.

Folie 15 / 50

Polling Server: feste Bediengelegenheiten

- Alle T beginnt eine Bediengelegenheit mit Budget Q.
- Ist die Queue dabei leer, verfällt diese Gelegenheit.
- Wird sie später leer, verfällt das Restbudget.
- Bei Budget null: bis zur nächsten Gelegenheit warten.

Warten auf einen Höherstörer verbraucht keine eigene CPU-Zeit.

Notizen 15 / 50

Ein Polling Server fragt zu regelmäßigen Freigaben nach wartender Arbeit. T ist der Abstand dieser Freigaben, Q die jeweils verfügbare Kapazität. Die Queue ist die Auftragswarteschlange. Ist sie beim Nachsehen leer, wird das Budget für diese Bediengelegenheit nicht für einen späteren Auftrag aufbewahrt. Sind Aufträge vorhanden, werden sie innerhalb des Budgets bedient. Wird die Queue leer, geht der ungenutzte Rest ebenfalls verloren. Bleiben Aufträge nach Budgeterschöpfung übrig, warten sie auf die nächste Freigabe.
CPU bedeutet Central Processing Unit, Prozessor. Die Kapazität zählt nur eigene CPU-Zeit. Unterbrechung durch eine höher priorisierte Aufgabe ist deshalb kein Leerlaufen der Queue und kein eigener Verbrauch.

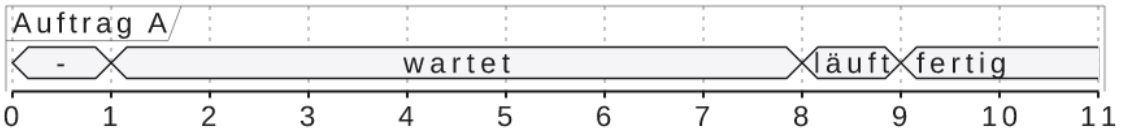
Notizen, Fortsetzung 15 / 50

Für die periodische Einplanung setzen wir voraus, dass eine volle Bediengelegenheit bis zur nächsten Freigabe abgearbeitet werden kann. Restbudget wird nicht über mehrere Perioden angespart. Dieses Verfahren ist leichter mit einem periodischen Task-Modell einzuplanen, reagiert jedoch empfindlich darauf, ob ein Auftrag kurz vor oder kurz nach dem Nachsehen ankommt.

Folie 16 / 50

Polling verpasst den Auftrag kurz nach null

Gemeinsames Modell: $Q = 2, T = 8$ ms · A$(1, 1)$, B$(6, 1)$.



A wartet 1–8: **7 ms bis zur Gelegenheit**.
Abschluss bei 9: Antwortzeit $R_A = 9 - 1 = 8$ ms.

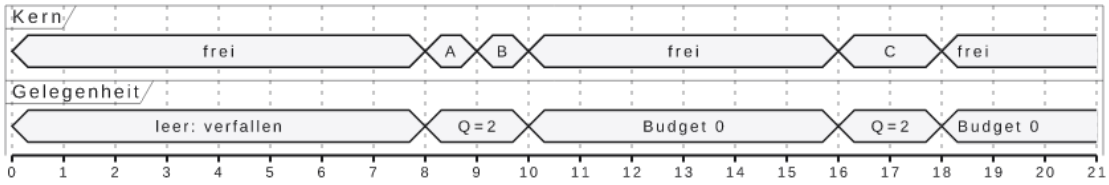
Notizen 16 / 50

Q ist die Serverkapazität von zwei Millisekunden, T die Periode von acht Millisekunden. Im gemeinsamen Modell kommt A bei eins mit Bedarf eins und B bei sechs mit Bedarf eins. Die Auftragsqueue ist beim Pollingzeitpunkt null leer. Nach der definierten Regel verfällt damit die erste Bediengelegenheit. CPU bedeutet Central Processing Unit, Prozessor. A darf trotz freiem Prozessor nicht bei eins starten; das wäre eine andere Serverregel.
Bei sechs wird B hinter A eingereiht. FIFO heißt First In, First Out, zuerst eingegangen zuerst bedient. Erst bei acht wird das Budget auf zwei gesetzt und der Server erneut freigegeben. A rechnet von acht bis neun und B von neun bis zehn. Ein bei neun ankommender dritter Auftrag kann eingereiht werden, erhält aber nach diesen beiden Einheiten keine zusätzliche Kapazität. Dass ein System scheinbar ungenutzte Zeit enthält, widerspricht dem Modell nicht: Die Regel reserviert eine bestimmte Art der Bedienung und schließt zusätzliche Hintergrundbedienung hier bewusst aus. Eine Kombination müsste gesondert benannt werden.
Die zusätzliche Zustandsspur verfolgt nur Auftrag A; die waagerechte Achse zählt Millisekunden. Der Strich zeigt: A ist bis eins noch nicht vorhanden. A wartet von eins bis acht, rechnet von acht bis neun und ist danach fertig. B bleibt Teil desselben gemeinsamen Modells und wird anschließend bedient. Die sieben Millisekunden bis zum nächsten Poll sind nur Wartezeit; zur vollständigen Antwortzeit gehören zusätzlich die eigene Arbeit und gegebenenfalls weitere Verzögerungen. Deshalb ist die Pollingperiode keine allgemeine Schranke für den Abschluss beliebiger Aufträge.

Folie 17 / 50

Polling: die vollständige Vergleichszeitachse

$Q = 2, T = 8$ ms · A$(1, 1)$, B$(6, 1)$, C$(9, 2)$



Abschluss A: 9 · B: 10 · C: 18 ms.

Notizen 17 / 50

Die horizontale Achse des PlantUML-Diagramms zeigt Millisekunden. Die obere Spur benennt die jeweils rechnende Arbeit oder den freien Kern. Die untere markiert Bediengelegenheiten und ihre Budgetfolgen. $Q = 2$ ist die Kapazität, $T = 8$ die Periode. Die Schreibweise A$(1, 1)$ bedeutet Freigabe eins und Bedarf eins; entsprechend hat B Freigabe sechs und Bedarf eins, C Freigabe neun und Bedarf zwei. Anfangs ist keine Arbeit vorhanden, daher verfällt die Gelegenheit bei null.
Bei acht läuft A eine Millisekunde. Bei neun sind B und der gerade eingetroffene Auftrag C vorhanden; FIFO, First In, First Out, gibt dem älteren B den Vorrang. Nach B bei zehn ist die Kapazität erschöpft. C wartet bis sechzehn und rechnet dann ohne Störung bis achtzehn. Die Antwortzeiten sind für A neun minus eins gleich acht, für B zehn minus sechs gleich vier und für C achtzehn minus neun gleich neun Millisekunden. Dies sind Ergebnisse dieses konkreten Modells, keine allgemeinen Maxima für beliebige Ankünfte.

Folie 18 / 50

Polling: verständlicher Preis der Einfachheit

- Eine leere Bediengelegenheit kann fast T Wartezeit auslösen.
- Frühere Aufträge können weitere Gelegenheiten belegen.
- Q/T allein begrenzt keine beliebige Auftragsantwortzeit.
- Vorteil: periodische Einplanung unter klaren Annahmen.

Notizen 18 / 50

T ist der Abstand der Polling-Gelegenheiten, Q ihre Kapazität. Kommt ein Auftrag unmittelbar nach einem leeren Pollingzeitpunkt an, kann er fast eine volle Periode auf den nächsten Beginn warten. Diese Aussage betrifft zunächst die nächste Gelegenheit, nicht schon den Abschluss. Ein großer Auftrag kann mehrere Budgets benötigen. Auch eine längere Warteschlange älterer Aufträge vergrößert die Antwortzeit. Der Quotient Q/T ist der nominelle Prozessoranteil des Servers, aber keine Frist für jeden Auftrag in seiner Queue. Eine Antwortzeitschranke für einzelne Aufträge braucht zusätzlich eine Grenze der eingehenden Arbeit und ihre Bedienreihenfolge. Der Vorteil des Polling Servers liegt in seiner regelmäßigen Freigabe:

Notizen, Fortsetzung 18 / 50

Wird jede Bediengelegenheit mit höchstens Q eigener Prozessorzeit begrenzt und kann sie rechtzeitig abgeschlossen werden, lässt sich ihre Störung konservativ wie die einer periodischen Aufgabe behandeln. Wir benutzen dafür nun die Antwortzeitanalyse aus Vorlesung 2; die Aussage wird später nicht ungeprüft auf den Deferrable Server übertragen.

Folie 19 / 50

Polling sauber in die bekannte RTA aufnehmen

Ein Kern · feste Prioritäten · vollständige Präemption.

- Periodische Aufgaben mit $D_i = T_i$, keine Sperrwartezeit
- Keine Freigabeschwankungen, Kosten hier null
- Polling Server als Aufgabe mit $C_S = Q, T_S = T$
- Volles Budget wird für den Nachweis angenommen.

RTA: Response-Time Analysis, Antwortzeitanalyse.

Notizen 19 / 50

RTA steht für Response-Time Analysis, Antwortzeitanalyse. Sie berechnet unter festen Prioritäten eine obere Antwortzeit. C_i ist der eigene Bedarf, T_i die Periode und D_i die relative Deadline der Aufgabe i. Hier gilt $D_i = T_i$, es gibt genau einen Kern, die Ausführung kann vollständig zugunsten höherer Prioritäten unterbrochen werden, und weder gemeinsame Sperren noch Freigabeschwankungen oder Verwaltungskosten sind angesetzt.
Beim Polling Server wird zu jeder regelmäßigen Gelegenheit höchstens Q eigene Arbeit verfügbar. Für den Nachweis ersetzen wir eine eventuell leere oder kürzere Bediengelegenheit konservativ durch den vollen Bedarf $C_S = Q$ bei Periode $T_S = T$. Der Index S bezeichnet den Server. Wichtig ist dabei, dass die volle Bediengelegenheit selbst bis zur nächsten Freigabe passt. Dann entstehen keine unmodellierten überlappenden Serverjobs. Dieser Ansatz weist die periodischen Aufgaben einschließlich der vollen Servergelegenheit nach. Er weist noch nicht die Antwortzeit jedes aperiodischen Auftrags nach, der möglicherweise mehrere Gelegenheiten in der Queue wartet.

Folie 20 / 50

Ein kleines nachweisbares Polling-Taskset

Aufgabe	Bedarf	Periode = Frist	Priorität
Regler	1	4	hoch
Polling Server	2	8	mittel
Diagnose	2	16	niedrig

$$U = \frac{1}{4} + \frac{2}{8} + \frac{2}{16} = \frac{5}{8} = 62{,}5\,\%$$

Alle Zeiten in ms · RMS-Rangfolge.

Notizen 20 / 50

Die Zahlen sind didaktisch vorgegeben und in Millisekunden angegeben. RMS bedeutet Rate Monotonic Scheduling: Die kürzere Periode erhält die höhere feste Priorität. Der Regler hat eigenen Bedarf eins, Periode und relative Frist vier. Der Polling Server hat Kapazität zwei, Periode und relative Frist acht. Die periodische Diagnoseaufgabe hat Bedarf zwei, Periode und Frist sechzehn. Diagnose bezeichnet hier diese eigene periodische Aufgabe, nicht einen Auftrag in der Serverqueue.
Die Gesamtauslastung U ist die Summe der Bedarfs-Perioden-Quotienten: ein Viertel plus zwei Achtel plus zwei Sechzehntel ergibt ein Viertel plus ein Viertel plus ein Achtel gleich fünf Achtel, also 62,5 Prozent. Diese Zahl beschreibt den langfristigen Rechenanteil im Modell. Die konkrete Antwortzeitanalyse auf der folgenden Folie zeigt zusätzlich, wie die höher priorisierten Jobs innerhalb eines Antwortzeitfensters stören. Für den Server nehmen wir auch dann zwei Einheiten an, wenn seine Queue in einem echten Lauf leer wäre. Das macht die Störung im Nachweis konservativ.

Folie 21 / 50

Polling-RTA: drei kurze Fixpunktrechnungen

$$R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j$$

Aufgabe	Iteration ab eigenem Bedarf	Ergebnis
Regler	1	$R_R = 1 \le 4$
Server	$2 \to 3 \to 3$	$R_S = 3 \le 8$
Diagnose	$2 \to 5 \to 6 \to 6$	$R_D = 6 \le 16$

Notizen 21 / 50

RTA bedeutet Response-Time Analysis, Antwortzeitanalyse. $R_i^{(k)}$ ist die Antwortzeitschätzung für Aufgabe i im Schritt k, C_i ihr eigener Bedarf. $hp(i)$ ist die Menge der höher priorisierten Aufgaben. T_j und C_j sind deren Perioden und Bedarfe. Die Aufrundung zählt, wie viele störende Jobs in das aktuelle Fenster passen. Ein Fixpunkt ist erreicht, wenn der nächste Wert unverändert bleibt.
Der Regler hat keine höhere Aufgabe und deshalb Antwortzeit eins. Für den Server ergibt sich ab zwei: $2 + \lceil 2/4 \rceil \cdot 1 = 3$; erneut eingesetzt bleibt drei. Für Diagnose starten wir bei zwei. Es folgt $2 + \lceil 2/4 \rceil \cdot 1 + \lceil 2/8 \rceil \cdot 2 = 5$. Mit fünf erhalten wir $2 + 2 \cdot 1 + 1 \cdot 2 = 6$; mit sechs bleibt sechs. Alle Zeiten sind Millisekunden. Die drei Ergebnisse liegen unter den relativen Fristen vier, acht und sechzehn. Damit ist auch die volle Polling-Gelegenheit vor ihrer nächsten Freigabe abgeschlossen. Eine aperiodische Anfrage, die diese Gelegenheit verpasst, kann trotzdem viel länger als drei warten; R_S ist nicht automatisch ihre eigene Antwortzeit.

Folie 22 / 50

Deferrable Server: Budget bis zur Grenze aufheben

- An festen Grenzen $0, T, 2T, \ldots$: Restbudget auf Q setzen.
- Bei leerer Queue bleibt ungenutztes Budget verfügbar.
- Ein späterer Auftrag kann es sofort benutzen.
- An der nächsten Grenze gibt es höchstens wieder Q.

Kein unbegrenztes Ansparen, kein Zusatzbudget pro Ankunft.

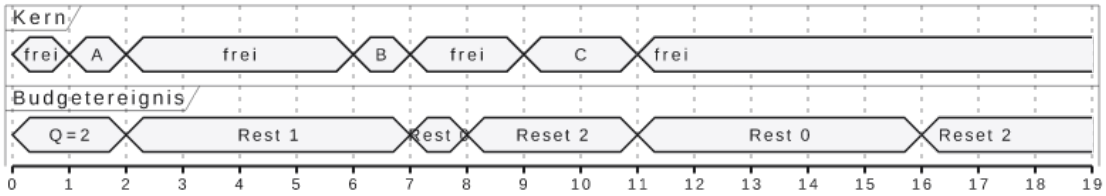
Notizen 22 / 50

Deferrable bedeutet aufschiebbar. Der Deferrable Server darf seine Kapazität innerhalb einer festen Periode für später eintreffende Arbeit aufheben. T ist die Periode der festen Auffüllgrenzen, Q die Kapazität und die Queue die Auftragswarteschlange. An jeder Grenze wird das Restbudget auf Q gesetzt, nicht um Q erhöht. Wer kurz vorher noch eine Einheit übrig hatte, besitzt danach bei $Q = 2$ zwei Einheiten und nicht drei.
Bei leerer Queue rechnet der Server nicht, verliert aber sein verbleibendes Budget nicht. Ein späterer Auftrag kann dieses Restbudget verwenden, sofern keine höher priorisierte Aufgabe läuft. Gezählt wird weiterhin nur die eigene Prozessorarbeit. Jede Auftragsankunft setzt das Budget also keinesfalls neu auf die volle Kapazität. Die Regel verbessert oft die erste Reaktion gegenüber Polling. Ihr Preis wird sichtbar, wenn ein spät benutztes Budget unmittelbar vor der festen Grenze und das neue Budget unmittelbar danach ausgeführt werden. Dieser Fall verändert die Störung niedriger priorisierter Aufgaben und verlangt eine passende Analyse.

Folie 23 / 50

Deferrable: dieselben Aufträge früher fertig

$Q = 2, T = 8$ ms · A$(1, 1)$, B$(6, 1)$, C$(9, 2)$



Abschluss A: 2 · B: 7 · C: 11 ms.

Notizen 23 / 50

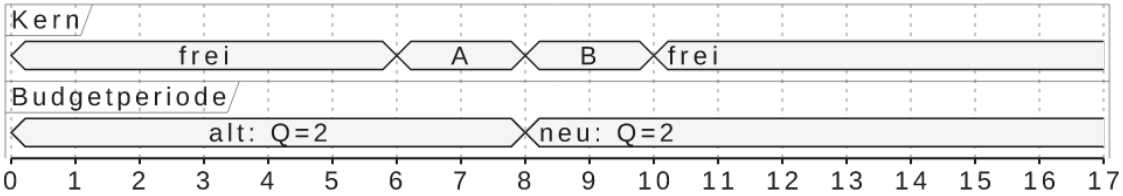
Die waagerechte Achse des PlantUML-Timing-Diagramms zeigt Millisekunden. Die Kernspur benennt rechnende Aufträge oder freie Zeit, die zweite Spur markiert Änderungen des Budgets. $Q = 2$ ist die Kapazität, $T = 8$ die feste Auffüllperiode. A$(1, 1)$ steht für Freigabe eins und Bedarf eins, B entsprechend sechs und eins, C neun und zwei. Höherstörer und Verwaltungskosten sind im gemeinsamen Modell ausgeschlossen.

A benutzt von eins bis zwei eine Einheit des Anfangsbudgets. Die verbliebene Einheit bleibt während der leeren Queue erhalten und versorgt B von sechs bis sieben. Danach beträgt das Restbudget null. Bei acht wird es unabhängig von den bisherigen Verbrauchszeiten wieder auf zwei gesetzt. C kommt bei neun an und rechnet bis elf durch. Die Antwortzeiten betragen A: zwei minus eins gleich eins, B: sieben minus sechs gleich eins und C: elf minus neun gleich zwei Millisekunden. Das bessere Ergebnis dieser Eingaben ist kein allgemeiner Dominanz- oder Fristnachweis. Besonders die Störung anderer Aufgaben muss nun genauer betrachtet werden.

Folie 24 / 50

Die Periodengrenze kann zwei Budgets verbinden

DS: $Q = 2, T = 8$ ms · A$(6, 2)$, B$(8, 2)$ · sonst keine Arbeit.



6–8: altes Budget · 8–10: neues Budget

Vier CPU-Millisekunden am Stück trotz $Q/T = 25\,\%$.

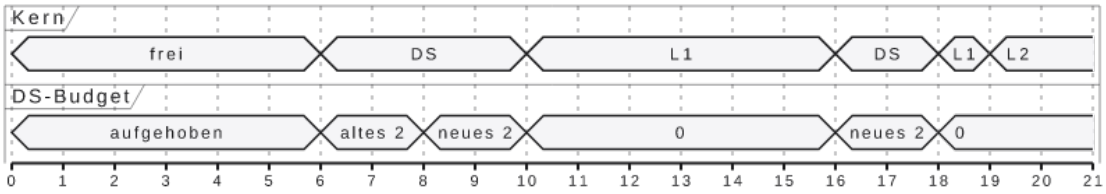
Notizen 24 / 50

Q ist die Kapazität des Deferrable Servers und T seine feste Auffüllperiode. Im neuen Beispiel ist das erste Budget bis sechs unbenutzt. A trifft bei sechs mit zwei Millisekunden eigenem Bedarf ein und darf von sechs bis acht rechnen. Bei acht endet dieses alte Budget, aber zugleich wird nach der Regel die Kapazität auf zwei gesetzt. B kommt genau bei acht mit Bedarf zwei an und rechnet bis zehn. Ankünfte und Auffüllungen werden vor der Auswahl berücksichtigt.
So entstehen vier aufeinanderfolgende Millisekunden Serverarbeit, obwohl Q nur zwei beträgt. Der langfristige nominelle Rechenanteil ist weiterhin zwei Achtel gleich 25 Prozent. Eine kurzfristige Störung kann daraus nicht durch schlichtes Multiplizieren eines Fensters mit 25 Prozent berechnet werden. Auch die Behandlung als einzelne gewöhnliche periodische Aufgabe mit Bedarf zwei und Periode acht kann eine solche verspätete alte plus neue Bedienung übersehen. Für Deferrable Server gibt es passende Einplanungstests und Interferenzschranken; die einfache Rechnung der Polling-Folien wird hier nicht übernommen.
Die Zeitachse zeigt oben die tatsächliche nacheinander ausgeführte Arbeit auf einem Kern und unten die Herkunft des Budgets. Der untere Zustandsname bezeichnet die jeweilige Budgetperiode, nicht den während der Rechnung kontinuierlich sinkenden Restzähler. Am gemeinsamen Zeitpunkt acht endet A, das Budget wird auf zwei gesetzt und B kann beginnen. Es gibt keine überlappenden Ausführungen; gerade die unmittelbar anschließenden Intervalle sechs bis acht und acht bis zehn bilden den Budgetstoß. Die Grenze acht ist keine zusätzliche Ausführungsdauer.

Folie 25 / 50

Warum die naive periodische RTA beim DS scheitert

DS: $Q = 2, T = 8$ · darunter L: $C_L = 7, T_L = D_L = 11$ ms



L freigegeben bei 6: Deadline 17, Abschluss 19, also $R_L = 13 > 11$.

Notizen 25 / 50

DS bedeutet Deferrable Server. Seine Kapazität $Q = 2$ wird alle $T = 8$ Millisekunden an festen Grenzen aufgefüllt. L ist eine niedriger priorisierte periodische Aufgabe mit eigenem Bedarf $C_L = 7$, Periode $T_L = 11$ und relativer Deadline $D_L = 11$. Die Rangfolge passt zu Rate Monotonic Scheduling: acht ist kürzer als elf. Serveraufträge mit je zwei Einheiten treffen bei sechs, acht und sechzehn ein. L hat seinen ersten Job bei sechs; sein nächster Job kommt bei siebzehn. Die Kernspur des PlantUML-Diagramms zeigt den ersten Job als L1 und ab neunzehn den bereits wartenden zweiten Job als L2; die zweite Spur erklärt die Serverbudgets. Die Achse zählt Millisekunden.
Der Server läuft sechs bis zehn und sechzehn bis achtzehn. L schafft sechs Einheiten von zehn bis sechzehn und die letzte von achtzehn bis neunzehn. Die Antwortzeit ist neunzehn minus sechs gleich dreizehn; seine absolute Deadline ist sechs plus elf gleich siebzehn. Eine naive Response-Time Analysis,

Notizen, Fortsetzung 25 / 50

Antwortzeitanalyse, mit dem Server als gewöhnlichem periodischen Bedarf liefert dagegen $7 \to 9 \to 11 \to 11$ und würde fälschlich Rechtzeitigkeit behaupten. Die Gesamtauslastung $2/8 + 7/11 = 39/44$ liegt sogar unter eins. Das Problem ist die ungeeignete Interferenzannahme, nicht ein Rechenfehler in der Aufrundung.

Folie 26 / 50

Sporadic Server: Verbrauch bestimmt die Rückgabe

- Ungenutztes Budget bleibt verfügbar.
- Verbrauchten Mengen werden Wiederauffüllungen zugeordnet.
- Rückgabezeit und Rückgabemenge stehen in einer Liste.
- Keine pauschale Rücksetzung an $0, T, 2T, \ldots$

Ziel: reaktionsfähige Ereignisbedienung und kontrollierte Störung.

Notizen 26 / 50

Sporadic Server bezeichnet das klassische Verfahren zur budgetierten Bedienung aperiodischer Arbeit. Aperiodisch bedeutet, dass die eingehenden Aufträge keinen zugesicherten Mindestabstand haben. T ist der zugeordnete Wiederauffüllzeitraum, aber keine allgemeine feste Resetfolge wie beim Deferrable Server. Nicht verbrauchte Kapazität bleibt verfügbar. Verbrauchte Kapazität wird nach den Regeln des Servers zu späteren Zeitpunkten zurückgegeben. Eine Wiederauffüllliste hält dafür Zeitpunkt und Menge fest.

Der klassische Algorithmus wurde für eine kontrollierte Einplanung zusammen mit periodischen Aufgaben entwickelt. Diese Eigenschaft entsteht durch seine vollständigen Aktivierungs-, Verbrauchs- und Wiederauffüllregeln. Sie folgt nicht allein aus dem Quotienten von Kapazität und Zeitraum. Besonders bei höher priorisierter Arbeit und überlappenden Auffüllungen ist die richtige Buchhaltung anspruchsvoll. Für den Einstieg rechnen wir deshalb einen klar bezeichneten klassischen Spezialfall vollständig aus: keine Höherstörer, ununterbrochene aktive Serverabschnitte und keine weitere Rückgabe innerhalb eines solchen Abschnitts. So wird das echte Rückgabeprinzip sichtbar, ohne eine vereinfachte Regel fälschlich als vollständigen Algorithmus auszugeben.

Folie 27 / 50

Sporadic Server: ein klassischer Spezialfall

Keine Höherstörer; jeder aktive Laufabschnitt ist ununterbrochen.

1. Abschnitt beginnt bei t_a mit verfügbarer Kapazität.
2. Server rechnet, solange Arbeit und Budget vorhanden sind.
3. Beim Leerlauf oder bei Budget null: verbrauchte Menge q buchen.
4. Diese Menge wird bei $t_a + T$ zurückgegeben.

In diesen Beispielen: keine weitere Rückgabe während eines Laufabschnitts.

Notizen 27 / 50

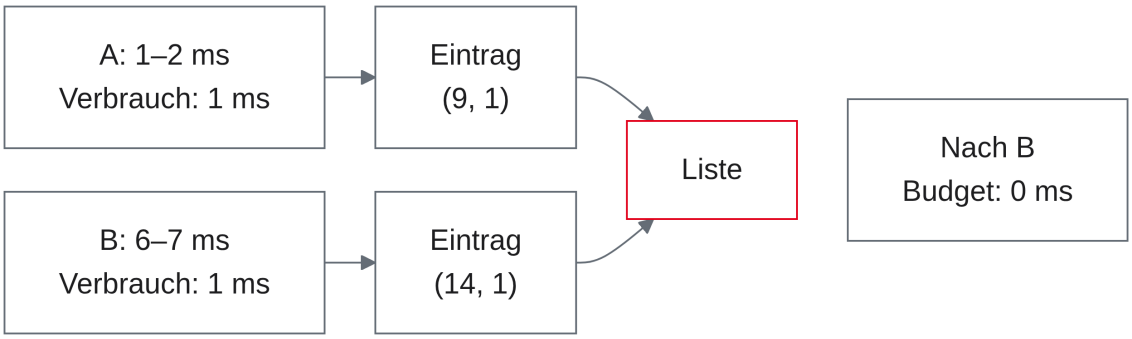
t_a ist der Beginn eines aktiven Serverabschnitts, T der Wiederauffüllzeitraum und q die in diesem Abschnitt tatsächlich verbrauchte eigene Prozessorzeit. Ohne höher priorisierte Störer ist der Abschnitt hier ununterbrochen, deshalb entspricht q seiner Länge. Er endet, wenn keine Aufträge mehr anstehen oder das verfügbare Budget null ist. Unbenutztes Restbudget bleibt erhalten. Die verbrauchte Menge wird als Paar aus Rückgabezeit $t_a + T$ und Menge q in die Liste aufgenommen. Eine fällige Rückgabe erhöht das Budget um genau ihre vorgemerkte Menge.

Ein Abschnitt ist nicht automatisch ein einzelner Auftrag: Unmittelbar anschließende wartende Aufträge können zum selben aktiven Abschnitt gehören. Ein Auftrag kann umgekehrt wegen Budgeterschöpfung mehrere Abschnitte benötigen. Alle gezeichneten Fälle vermeiden eine zusätzliche Rückgabe während laufender Serverarbeit. Das sind klassische eindeutige Fälle des Sporadic-Server-Prinzips, kein vollständiger Zustandsautomat für beliebige Präemptionen. Insbesondere darf bei höher priorisierter Arbeit der Aktivierungsbeginn nicht ungeprüft auf den ersten tatsächlichen Serverbefehl gelegt werden. Dort muss die vollständige klassische Regel des prioritätsbezogenen aktiven Intervalls verwendet werden.

Folie 28 / 50

Verbrauch erzeugt Listeneinträge

Gemeinsames Modell: $Q = 2, T = 8$ ms.



Kein Reset bei 8 ms · Rückgaben bei 9 und 14 ms.

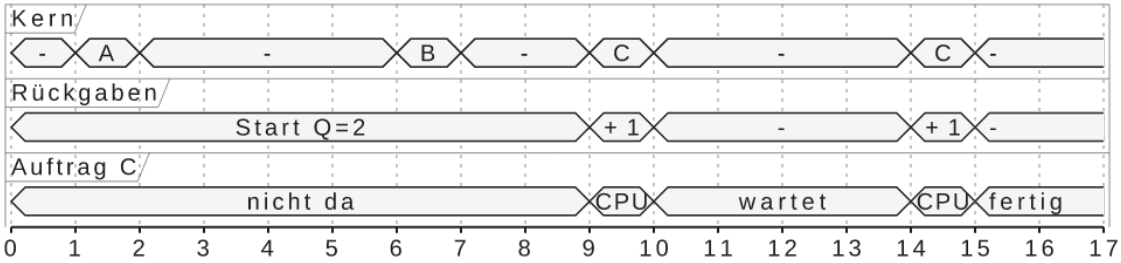
Notizen 28 / 50

Q ist die Kapazität von zwei Millisekunden und T der Wiederauffüllzeitraum von acht Millisekunden. Anfangs ist das Budget vollständig verfügbar. A kommt bei eins mit Bedarf eins, läuft von eins bis zwei und lässt eine Einheit unbenutztes Budget zurück. Sein aktiver Abschnitt beginnt bei $t_a = 1$ und verbraucht $q = 1$. Deshalb entsteht der Listeneintrag $(1 + 8, 1) = (9, 1)$. Das Paar bedeutet Rückgabezeit neun und Menge eins.
Während der leeren Queue von zwei bis sechs bleibt die andere Budgeteinheit erhalten. B benötigt sie von sechs bis sieben. Sein Abschnitt beginnt bei sechs und erzeugt entsprechend $(6 + 8, 1) = (14, 1)$. Die Einträge werden nach den Abschnittsenden bei zwei und sieben gebucht; neun und vierzehn sind ihre Fälligkeiten. Danach ist das aktuelle Budget null, aber beide verbrauchten Mengen sind in der Liste für spätere Rückgaben vermerkt. Bei acht passiert keine periodische Auffüllung. Diese Unterscheidung erklärt, warum ein späterer Auftrag beim Sporadic Server anders bedient wird als beim Deferrable Server, obwohl beide mit derselben Anfangskapazität und demselben Zeitraum arbeiten.
Die Pfeile zeigen von links nach rechts eine Buchungsabhängigkeit: Aus jedem aktiven Abschnitt entsteht genau ein Eintrag, der anschließend in der Rückgabeliste steht. Beim Anfangsbudget von zwei Millisekunden und dem Wiederauffüllzeitraum von acht Millisekunden liefert A den Zeitpunkt eins plus acht gleich neun mit Menge eins; B liefert sechs plus acht gleich vierzehn mit Menge eins. Jedes Paar nennt zuerst den Rückgabezeitpunkt und dann die Menge, jeweils in Millisekunden. Der separate Kasten nennt das bereits auf null gesunkene Budget nach B. Die Grafik ist keine maßstäbliche Zeitachse; ihre Pfeile bedeuten weder unmittelbar folgende Ausführung noch sofortige Budgetrückgabe. Es gilt der eingeführte klassische Spezialfall ohne höher priorisierte Arbeit und ohne Rückgabe innerhalb eines laufenden Abschnitts.

Folie 29 / 50

SS: C wartet trotz freiem Kern

$Q = 2, T = 8$ ms · A$(1, 1)$, B$(6, 1)$, C$(9, 2)$



C rechnet 9–10 und 14–15 · $R_C = 15 - 9 = 6$ ms.

Notizen 29 / 50

Die waagerechte Achse des PlantUML-Diagramms zeigt Millisekunden. Die obere Spur nennt rechnende Aufträge; ein Strich bedeutet einen freien Kern. Die mittlere Spur nennt das Anfangsbudget und markiert die Rückgabeereignisse mit +1; der Strich bedeutet keine weitere Rückgabe. Sie zeigt nicht das gesamte aktuelle Budget. Die späteren Rückgaben bei siebzehn und zweiundzwanzig liegen außerhalb des dargestellten Zeitraums und stehen in der folgenden Tabelle. $Q = 2$ ist die Anfangskapazität, $T = 8$ der Wiederauffüllzeitraum. Die Auftragsschreibweise gibt jeweils Freigabe und Bedarf an. Es gibt keine höher priorisierte Aufgabe und keine zusätzliche Rückgabe innerhalb eines laufenden Serverabschnitts.

Notizen, Fortsetzung 29 / 50

A rechnet eins bis zwei, B sechs bis sieben. Ihre jeweiligen Rückgaben von einer Einheit sind bei neun und vierzehn fällig. C kommt bei neun mit zwei Einheiten Bedarf an. Die gerade zurückgegebene eine Einheit reicht von neun bis zehn; danach fehlt Budget. Der Prozessor bleibt in unserem Modell von zehn bis vierzehn frei, obwohl C noch nicht fertig ist. Erst die Rückgabe bei vierzehn erlaubt seine letzte Einheit bis fünfzehn. Der eigene Bedarf C_C beträgt zwei und die Antwortzeit R_C sechs Millisekunden. Der Abschnitt neun bis zehn erzeugt die Rückgabe bei siebzehn; der Abschnitt vierzehn bis fünfzehn jene bei zweiundzwanzig. Diese späteren Einheiten bleiben ohne neue Aufträge ungenutzt verfügbar.
Die neue dritte Spur macht den Zustand von C ausdrücklich sichtbar. Zwischen zehn und vierzehn ist C noch nicht fertig und wartet auf Budget, obwohl die Kernspur frei meldet. Zwischen neun und zehn sowie vierzehn und fünfzehn läuft C; dafür steht in der Zustandsspur CPU, also die Zuteilung des Prozessors. Ab fünfzehn ist es fertig. Die mittlere Spur nennt Rückgabeereignisse, die untere den Auftragszustand. Ein Rückgabeereignis ist damit weder ein neuer Auftrag noch automatisch dessen Fertigstellung.

Folie 30 / 50

Sporadic Server: Budget und Liste zusammen lesen

Unmittelbar nach dem jeweiligen Ereignis · Zeiten in ms.

Zeit	Ereignis	Budget b	noch vorgemerkt
7	B fertig	0	$(9, 1), (14, 1)$
9	Rückgabe, C startet	1	$(14, 1)$
10	erster C-Teil endet	0	$(14, 1), (17, 1)$
14	Rückgabe, C setzt fort	1	$(17, 1)$
15	C fertig	0	$(17, 1), (22, 1)$

Notizen 30 / 50

b ist das verfügbare Restbudget des Servers. Jedes Listenpaar nennt zuerst den Rückgabezeitpunkt und dann die Menge. Die Tabelle trennt eine punktförmige Rückgabe am Beginn eines Abschnitts von dem anschließenden Verbrauch über dessen Dauer. Bei neun wird eine Einheit zurückgegeben und C beginnt zu rechnen. Direkt zu diesem Zeitpunkt ist das Budget eins; es sinkt während des Abschnitts neun bis zehn auf null. Es wird nicht schon bei der Auswahl des Auftrags als verstrichene CPU-Zeit abgebucht. CPU bedeutet Central Processing Unit, Prozessor.
Nach Ende des ersten C-Abschnitts steht die gerade verbrauchte Einheit als Rückgabe bei siebzehn in der Liste. Bei vierzehn wird der ältere Eintrag entfernt und das Budget um eins erhöht. Die letzte C-Einheit verbraucht diese Kapazität bis fünfzehn und erzeugt $(22, 1)$. In den Zeilen nach Abschnittsenden ist die Summe aus verfügbarem Budget und vorgemerkten Mengen wieder die Gesamtkapazität zwei. Während eines laufenden Abschnitts muss der bereits laufend erfasste, noch nicht abschließend eingetragene Verbrauch zusätzlich berücksichtigt werden. Die Liste ist dynamische Buchhaltung und keine Liste der Auftragsdeadlines.

Folie 31 / 50

Gleiche Kapazität, unterschiedliche Antworten

Gemeinsame Eingaben · Abschlüsse und Antwortzeiten in ms.

Verfahren	Ende A / B / C	Antwort A / B / C
Background ohne Vordergrund	2 / 7 / 11	1 / 1 / 2
Polling	9 / 10 / 18	8 / 4 / 9
Deferrable	2 / 7 / 11	1 / 1 / 2
Sporadic	2 / 7 / 15	1 / 1 / 6

Der einzelne Vergleich beweist keinen allgemeinen Sieger.

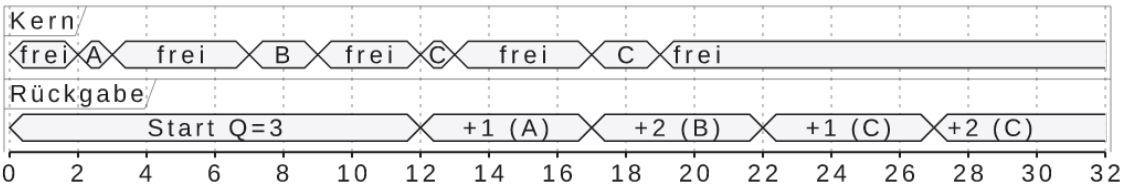
Notizen 31 / 50

Alle Werte beziehen sich auf die Freigaben A bei eins, B bei sechs und C bei neun mit eigenen Bedarfen eins, eins und zwei. Die Antwortzeit eines Auftrags ist sein Abschluss minus seine Freigabe. Beim Polling ergibt sich deshalb acht, vier und neun; beim Deferrable Server eins, eins und zwei; beim Sporadic Server eins, eins und sechs. Dessen Rechnung ist der vollständig definierte klassische Spezialfall ohne Höherstörer. Background ist Hintergrundbearbeitung mit niedrigster Priorität und ohne reserviertes Budget. Im gemeinsamen Modell fehlt jede Vordergrundarbeit; deshalb kann Background die drei Aufträge sofort bedienen.
Für die budgetierten Verfahren sind Kapazität und Zeitraum dieselben: zwei und acht Millisekunden. Der unterschiedliche Zeitpunkt der Verfügbarkeit erzeugt die verschiedenen Ergebnisse. Dass Background und Deferrable hier gleich aussehen, macht sie nicht zu demselben Verfahren. Mit Vordergrundlast kann Background stark verzögert werden. Dass Deferrable hier die kürzesten Antworten hat, beantwortet nicht die Fristen der verdrängten periodischen Aufgaben. Ein allgemeiner Vergleich benötigt Lastannahmen und passende Nachweise für alle zu schützenden Aufgaben.

Folie 32 / 50

SS: gebündelte Rückgaben

$Q = 3, T = 10$ ms · A$(2, 1)$, B$(7, 2)$, C$(11, 3)$ · keine Höherstörer.



B rechnet 7–9: **eine gebündelte Rückgabe** $(17, 2)$.
C: $C_C = 3$ ms, $R_C = 19 - 11 = 8$ ms.

Notizen 32 / 50

SS steht für Sporadic Server. $Q = 3$ ist die Kapazität und $T = 10$ der Wiederauffüllzeitraum. FIFO bedeutet First In, First Out, Bedienung nach Ankunft. Wie im ersten Beispiel gibt es keine höher priorisierten Aufgaben und keine weitere Rückgabe innerhalb eines laufenden Serverabschnitts. Die Zahlen sind neue didaktische Vorgaben, damit die Regel unabhängig vom ersten Beispiel nachvollzogen werden kann.
A rechnet zwei bis drei, verbraucht eins und erzeugt die Rückgabe $(2+10, 1) = (12, 1)$. Zwei Einheiten bleiben verfügbar. B benutzt diese von sieben bis neun und erzeugt eine gebündelte Rückgabe von zwei bei siebzehn. Es sind ausdrücklich nicht zwei getrennte Quantenrückgaben bei siebzehn und achtzehn. C kommt bei elf an, trifft auf Budget null und wartet bis zwölf. Es rechnet eine Einheit bis dreizehn, wartet auf die Rückgabe von zwei bei siebzehn und endet nach zwei weiteren Einheiten bei neunzehn. Seine Abschnitte erzeugen $(22, 1)$ und $(27, 2)$. C benötigt insgesamt drei eigene Millisekunden; fünf weitere Millisekunden entfallen auf Warten. Der größere Zeitraum zwischen Freigabe und Ende wird nicht als Budgetverbrauch abgerechnet.
Oben wird die CPU-Belegung von null bis dreißig Millisekunden gelesen; unten stehen die Rückgabeereignisse mit Menge und Herkunft. Die untere Beschriftung bleibt bis zum nächsten Ereignis stehen und ist keine Darstellung des verfügbaren Restbudgets. B rechnet zusammenhängend zwei Millisekunden. Am Abschnittsende bei neun wird deshalb der einzige Eintrag $(17, 2)$ gebucht; seine Rückgabe ist bei siebzehn fällig. C kann bei siebzehn beide Einheiten nacheinander nutzen und endet bei neunzehn. Die späteren Rückgaben bei zweiundzwanzig und siebenundzwanzig bleiben bei leerer Auftragsqueue ungenutzt verfügbar. Wie zuvor gibt es keine höher priorisierte Arbeit und keine zusätzliche Rückgabe innerhalb eines aktiven Abschnitts.

Folie 33 / 50

Grenzen der Einstiegsregel

- Höherpriorisierte Arbeit verändert die aktiven Intervalle.
- Rückgaben können mit laufender Bedienung überlappen.
- Mehrere Mengen benötigen korrekte Listenverwaltung.
- Zusammenlegen und Begrenzen der Liste brauchen feste Regeln.

Die klassischen Einplanungseigenschaften gelten für den korrekt spezifizierten Algorithmus.

Notizen 33 / 50

SS bedeutet Sporadic Server. Die bisherigen vollständigen Rechnungen sind bewusst auf eindeutige ununterbrochene aktive Serverabschnitte ohne Höherstörer begrenzt. Im allgemeinen klassischen Verfahren ist ein prioritätsbezogenes aktives Intervall zu betrachten: Auch höher priorisierte Ausführung kann dazugehören. Sein Beginn darf deshalb nicht pauschal mit dem ersten tatsächlichen Serverbefehl gleichgesetzt werden. Ebenso erfordern Rückgaben während laufender Bedienung und begrenzte Listen eigene Regeln. Ein beliebiges Zusammenlegen von Mengen kann zu früher Kapazität freigeben und die Einplanungseigenschaft zerstören.
Die klassische Theorie des Sporadic Servers von Sprunt, Sha und Lehoczky verfolgt die Kompatibilität mit der Einplanung periodischer Aufgaben. Das ist eine Eigenschaft des genau definierten Verfahrens unter seinen Voraussetzungen, keine Folge der Namenswahl und kein Beweis allein durch Kapazität geteilt durch Zeitraum. Wir leiten hier weder eine allgemeine Serverantwortzeit noch eine vollständige

Notizen, Fortsetzung 33 / 50

präemptive Implementierung aus der Einstiegsregel ab. Eine eigenständig erdachte Rückgabe jedes Ein-Millisekunden-Quantums wäre ein anderes, eventuell konservatives Lehrmodell. Sie wird hier nicht mit dem vollständigen klassischen Algorithmus gleichgesetzt.

Folie 34 / 50

Leere Queue und leeres Budget unterscheiden

Lage	Polling	Deferrable / Sporadic
Queue wird leer	Restbudget verfällt	Restbudget bleibt
Budget wird null	spätere Gelegenheit	nächste passende Auffüllung
Budget positiv, Höherstörer läuft	eigene CPU-Zeit bleibt unbenutzt	eigene CPU-Zeit bleibt unbenutzt

Eine Warteursache sagt noch nichts über die Auftragsfrist.

Notizen 34 / 50

Die Queue ist die Warteschlange der noch nicht abgeschlossenen Aufträge, das Budget die noch erlaubte eigene Prozessorzeit. Wird die Queue beim Polling Server leer, verfällt das Restbudget der aktuellen Gelegenheit. Beim Deferrable Server bleibt es bis zur nächsten festen Auffüllgrenze benutzbar; bei dieser Grenze wird auf die Kapazität gesetzt. Beim Sporadic Server bleibt ungenutztes Budget erhalten, während bereits verbrauchte Mengen nach seinen Regeln in der Rückgabeliste stehen.
CPU bedeutet Central Processing Unit, Prozessor. Eine höher priorisierte Aufgabe, auch Höherstörer genannt, verbraucht keine eigene Server-CPU-Zeit. Dass das Budget währenddessen nicht als eigene Rechenzeit sinkt, legt aber die vollständige Wiederauffüllbuchhaltung des Sporadic Servers noch nicht fest. Bei Budget null bleiben vorhandene Aufträge fachlich unerledigt. Sie verschwinden weder aus der Queue noch wird ihre Deadline automatisch verschoben. Die vier Verfahren regeln die Bedienung; ein Vertrag über erlaubte Verzögerung und den Umgang mit verspäteten Aufträgen muss zusätzlich existieren.

Folie 35 / 50

Auftragsdeadline und Serverzeitraum sind verschieden

Gemeinsamer Auftrag C: Freigabe $r_C = 9$, Bedarf $C_C = 2$ ms.
Angenommene relative Auftragsfrist: $D_C = 4$ ms, also $d_C = 13$ ms.

Verfahren	Abschluss	Frist 13 erfüllt?
Polling	18	nein
Deferrable	11	ja
Sporadic	15	nein

Notizen 35 / 50

r_C ist die Freigabe des Auftrags C, C_C sein eigener Bedarf, D_C seine relative Deadline und $d_C = r_C + D_C$ die absolute Deadline. Die Symbole mit Index C beziehen sich hier auf den Auftrag mit Namen C. Mit Freigabe neun und relativer Frist vier liegt die absolute Frist bei dreizehn Millisekunden. Die bisherigen Vergleichsrechnungen ergeben Abschlüsse achtzehn, elf und fünfzehn. Deshalb erfüllt nur der Deferrable-Verlauf diese konkrete neu hinzugefügte Frist.
Daraus folgt kein allgemeiner Vorteil für beliebige harte Fristen: Andere Eingaben und die Fristen anderer Tasks bleiben zu prüfen. Der Serverzeitraum von acht Millisekunden ist insbesondere nicht die relative Deadline jedes Auftrags. Er bestimmt je nach Verfahren Auffüllgrenzen oder Rückgabezeiten. Ein Fristvertrag für die Aufträge benötigt zusätzlich eine begrenzte Eingangsarbeitsmenge, die Reihenfolge innerhalb der Queue und eine passende Bedien- beziehungsweise Antwortzeitanalyse. Ein Auftrag kann eine Fristverletzung erleiden, obwohl der Server seine Budgetregel vollständig korrekt eingehalten hat.

Folie 36 / 50

Überlast: Budgetbegrenzung beseitigt keine Arbeit

- Eingehende Arbeit kann die mögliche Bedienung übersteigen.
- Das Budget begrenzt die CPU-Nutzung des Servers.
- Die Queue wächst oder eine Annahmeregel greift.
- Festlegen: warten, verwerfen, zusammenfassen, ablehnen.

$Q = 2, T = 8$: dauerhaft 4 ms Arbeit je 8 ms passen nicht in 25 %.

Notizen 36 / 50

Q ist die Serverkapazität und T sein Zeitraum. Zwei Millisekunden je acht ergeben nominell 25 Prozent Prozessoranteil. CPU bedeutet Central Processing Unit, Prozessor. Werden dauerhaft vier Millisekunden neue Arbeit je acht Millisekunden verlangt, übersteigt das diese Bedienkapazität. Selbst wenn der Server seinen möglichen Anteil vollständig erhält, kommen im Mittel zwei Millisekunden unerledigte Arbeit pro solchem Eingangsintervall hinzu. Höher priorisierte Belastung kann die Bedienung zusätzlich verzögern;

Notizen, Fortsetzung 36 / 50

für kurze Fenster gelten die jeweiligen Serverregeln.
Eine unbeschränkte Queue würde immer länger, eine endliche Queue wird irgendwann voll. Eine Annahmeregel bestimmt, was dann passiert: zum Beispiel einen Auftrag ablehnen, unwichtige alte Aufträge verwerfen oder mehrere gleichartige Zustandsmeldungen fachlich zulässig zusammenfassen. Solche Entscheidungen ändern die Anforderungserfüllung und müssen spezifiziert werden. Auch ein niedrigerer mittlerer Eingangswert allein garantiert keine kurze Antwortzeit, wenn beliebig große Auftragsstöße erlaubt sind. Die Begrenzung des Serververbrauchs schützt andere Arbeit nur unter dem angenommenen Modell; sie verspricht keine rechtzeitige Verarbeitung unbegrenzter Eingaben.

Folie 37 / 50

Temporale Isolation braucht die ganze Grenze

Budget korrekt begrenzen und passende Einplanung nachweisen.
Zusätzlich prüfen:

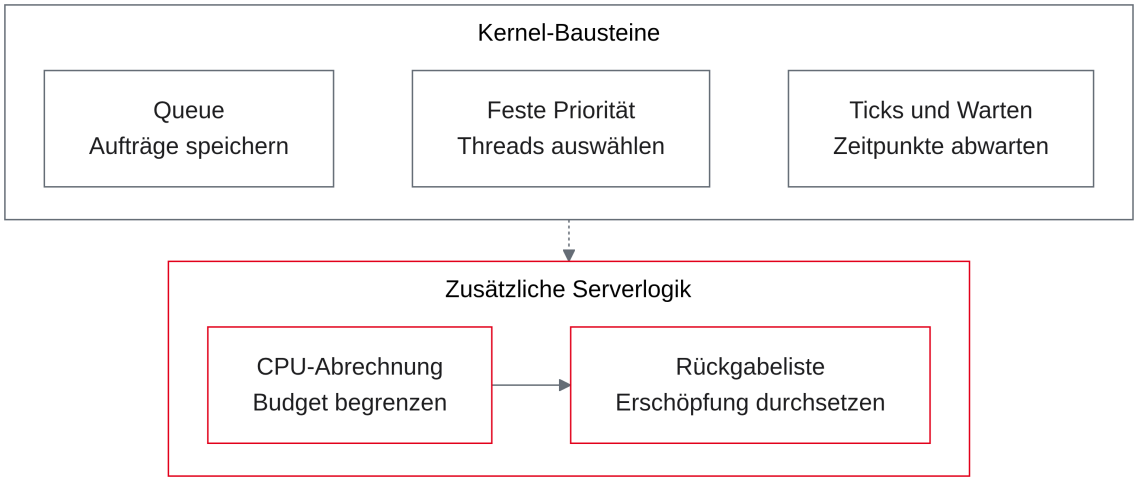
- Interrupt- und Queueaufwand außerhalb des Servers
- Gemeinsame Sperren und nicht unterbrechbare Abschnitte
- Bedarfsüberschreitungen und volle Queues
- Ausführung und Störungen je Kern

Notizen 37 / 50

Temporale Isolation bedeutet, die zeitlichen Auswirkungen eines Teilnehmers auf andere unter einem benannten Vertrag zu begrenzen. Eine korrekt erzwungene Budgetgrenze ist dafür ein Baustein. Zusätzlich muss das gewählte Serververfahren passend eingeplant sein. Beim Deferrable Server darf zum Beispiel der mögliche Budgetstoß an einer Periodengrenze nicht übersehen werden. Eine feste Priorität oder ein nomineller Prozentanteil allein begründet diese Isolation nicht.
Eine Interruptroutine bearbeitet ein Hardwareereignis und kann außerhalb des budgetierten Serverthreads laufen. Wenn jeder neue Auftrag bereits dort viel Arbeit auslöst, kann die Eingangsflut trotz begrenzter Serverrechnung den Kern belasten. Ebenso kann ein Server mit knappem Rechenbudget eine gemeinsame Sperre lange blockieren, wenn er sie über eine budgetbedingte Pause hält. Nicht unterbrechbare Abschnitte und tatsächliche maximale Bedarfe müssen im Modell auftauchen. Auf mehreren Kernen kommen mögliche gemeinsame Speicher- und Busstörungen hinzu. Die heutigen Einprozessorrechnungen werden deshalb nicht durch bloßes Zusammenzählen beider Kernkapazitäten zu einem Mehrkernnachweis.

Folie 38 / 50

FreeRTOS braucht Serverlogik



`osDelay()` allein misst und begrenzt keine eigene CPU-Zeit.

Notizen 38 / 50

FreeRTOS ist das im Kurs verwendete Echtzeitbetriebssystem. Ein Thread ist ein verwalteter Ausführungspfad mit eigenem gespeicherten Kontext; seine Priorität beeinflusst die Auswahl, sobald er bereit ist. Eine Queue ist eine Warteschlange, in der Aufträge bis zur Bearbeitung gespeichert werden können. Weder eine hohe Threadpriorität noch eine Queue aktiviert automatisch einen Polling, Deferrable oder Sporadic Server. Budgetprüfung, Wiederauffüllungen, Unterbrechung bei Erschöpfung und Überlastbehandlung sind zusätzliche Aufgaben der Umsetzung.
osDelay ist eine Wartefunktion der verwendeten CMSIS-RTOS2-Schnittstelle. CMSIS heißt Common Microcontroller Software Interface Standard, RTOS Real-Time Operating System, Echtzeitbetriebssystem. Der Aufruf blockiert den aufrufenden Thread für eine Anzahl Kernel-Ticks, also Zeitschritte der Betriebssystemzeitbasis. Er misst nicht die zuvor ausschließlich von diesem Thread verbrauchte CPU-Zeit; CPU bedeutet Central Processing Unit, Prozessor. Eine Schleife aus Rechnen und osDelay wird deshalb nicht allein durch die gewählten Zahlen zu einer korrekten Sporadic-Server-Implementierung. Der praktische Teil bezeichnet seine modellierten Größen ausdrücklich als solche.
Die Kernelbausteine benötigen zusätzliche Serverlogik. Die obere Gruppe nennt vorhandene Fähigkeiten: Aufträge speichern, bereite Threads nach fester Priorität auswählen und anhand der Kernelzeit warten. Der gestrichelte Pfeil zwischen den Gruppen bedeutet, dass die zusätzliche Anwendungslogik die Kernelbausteine verwendet; er behauptet keine bereits fertige Serverfunktion. Der durchgezogene Pfeil bezeichnet die Buchung tatsächlich verbrauchter eigener CPU-Zeit in die Rückgabeliste. Zusätzlich muss die Umsetzung Budgeterschöpfung rechtzeitig durchsetzen. Die räumliche Trennung der Gruppen zeigt Verantwortlichkeiten und keine Ausführungsreihenfolge. Insbesondere misst ein Warten in Kernel-Ticks keine eigene CPU-Zeit; eine Rückgabeliste und ein Timer allein ergeben keinen vollständigen präemptiven Sporadic Server.

Folie 39 / 50

Reale CPU-Abrechnung ist eine eigene Aufgabe

- Laufzeit nur während tatsächlicher Serverausführung belasten
- Präemption, Interrupts und Zeitauflösung korrekt behandeln
- Budgeterschöpfung rechtzeitig wirksam machen
- Rückgabeereignisse und Queuezugriffe nebenläufig schützen

Ein beobachtetes Funktionsintervall ist zunächst verstrichene Zeit.

Notizen 39 / 50

CPU bedeutet Central Processing Unit, Prozessor. Eine reale Budgetabrechnung muss die Zeit erkennen, in der der Server selbst tatsächlich auf dem Kern ausgeführt wird. Ein Zeitstempel vor und nach einer Funktion enthält dagegen auch Unterbrechungen durch andere Arbeit. Er ist zunächst eine Messung verstrichener Zeit. Je nach Zeitquelle und Einbindung kann auch eine Laufzeitstatistik fremde Interruptarbeit enthalten; die Bedeutung der verwendeten Messung muss deshalb geprüft werden.
Präemption ist die erzwungene Unterbrechung zugunsten höherer Priorität. Zusammen mit Timerauflösung und Verwaltungsaufwand beeinflusst sie, wie genau eine Budgeterschöpfung erkannt und durchgesetzt werden kann. Ein lange laufender ungeteilter Arbeitsschritt darf nicht erst weit nach Budgetende anhalten. Gleichzeitig eintreffende Aufträge und Rückgabeereignisse erfordern konsistente Änderungen gemeinsamer Daten und der Queue, also der Auftragswarteschlange. Diese Anforderungen erklären, warum eine vollständige präemptive Serverumsetzung anspruchsvoller ist als eine Papierregel. Der kurze Versuch in diesem Termin verwendet daher eine explizite virtuelle Zeit und bekannte endliche Arbeitseinheiten, statt einen allgemeinen Laufzeitbudgetmechanismus zu behaupten.

Folie 40 / 50

Ausblick: Server unter EDF

- Heute: feste Serverpriorität, getrennte Budgetregel
- EDF: Auswahl nach absoluter Deadline
- CBS: Constant Bandwidth Server für deadlinebasiertes Scheduling
- Dafür ändern sich Budget- und Deadlineverwaltung.

Die heutigen Regeln nicht ungeprüft in einen EDF-Scheduler übernehmen.

Notizen 40 / 50

EDF bedeutet Earliest Deadline First: Unter den bereiten Jobs erhält derjenige mit der frühesten absoluten Deadline den Prozessor. CBS steht für Constant Bandwidth Server, ein Serververfahren für die budgetierte Einplanung in einem solchen deadlinebasierten Umfeld. Dabei wird zusätzlich zur Kapazität eine Serverdeadline verwaltet. Die vollständigen Regeln und ihr Nachweis sind ein eigenes Vertiefungsthema. Die bisherige Vorlesung verwendet dagegen feste Prioritäten. Ein Auftrag in der Serverqueue verändert nicht einfach den festen Rang des Serverthreads. Seine eigene Deadline ist wiederum von Serverzeitraum

Notizen, Fortsetzung 40 / 50

und eventueller Serverdeadline zu unterscheiden. Der Ausblick zeigt, dass die Frage nach kontrollierter Ereignisbedienung unter verschiedenen Schedulern existiert. Er behauptet weder, dass FreeRTOS standardmäßig EDF verwendet, noch dass ein Constant Bandwidth Server durch Austausch einer Konstante aus den heutigen Regeln entsteht. In den folgenden Papieraufgaben bleiben wir vollständig bei den eingeführten festen Prioritäten und den ausdrücklich festgelegten Modellfällen.

Folie 41 / 50

Papier 1: Polling und Deferrable vergleichen

10 Minuten · neues Lehrbeispiel · alle Zeiten in ms.
$Q = 2, T = 6$ · keine Höherstörer · FIFO · Anfangsbudget 2.

Auftrag	Freigabe	Bedarf
A	1	1
B	5	2

Zeichnen Sie beide Zeitachsen und berechnen Sie die Antwortzeiten.

Notizen 41 / 50

Q ist die Kapazität und T der feste Abstand der Auffüllgrenzen. FIFO heißt First In, First Out, Bedienung nach Ankunft. Es gibt keine höher priorisierte Aufgabe und keine Verwaltungskosten. Bei einer gemeinsamen Zeitgrenze werden Ankünfte und Auffüllungen vor der Auswahl berücksichtigt. Die Antwortzeit ist Abschluss minus Freigabe. Diese Angaben machen die Aufgabe vollständig auf Papier lösbar.
Beim Polling Server ist die Queue bei null leer; die Gelegenheit verfällt. Bei sechs werden zwei Einheiten verfügbar. A rechnet sechs bis sieben, B sieben bis acht seine erste Einheit. Dann ist das Budget null. Bei zwölf erhält B die letzte Einheit und endet bei dreizehn. Die Antwortzeiten sind für A sieben minus eins gleich sechs und für B dreizehn minus fünf gleich acht. Beim Deferrable Server bleibt das Anfangsbudget für später verfügbar. A rechnet eins bis zwei, eine Einheit bleibt. B rechnet fünf bis sechs die erste Einheit; bei sechs wird auf zwei aufgefüllt, die zweite B-Einheit läuft sechs bis sieben. Die Antwortzeiten sind eins und zwei. Nach B bleibt beim Deferrable Server noch eine Einheit verfügbar. Typischer Fehler ist, beim Polling Server A schon bei eins laufen zu lassen.

Folie 42 / 50

Papier 2: die klassische SS-Rückgabe bündeln

10 Minuten · $Q = 2, T = 6$ ms · Anfangsbudget 2.
Keine Höherstörer; ununterbrochene aktive Abschnitte.

Auftrag	Freigabe	Bedarf
A	1	2
B	5	1

Zeitachse, Rückgabeliste und Budget direkt nach Ende B angeben.

Notizen 42 / 50

SS bedeutet Sporadic Server. $Q = 2$ ist die Kapazität und $T = 6$ der Wiederauffüllzeitraum in Millisekunden. Im klassischen Spezialfall gibt es keine höher priorisierten Störer und keine zusätzliche Rückgabe während eines laufenden Serverabschnitts. Der tatsächliche Verbrauch eines ununterbrochenen aktiven Abschnitts wird zum Abschnittsbeginn plus sechs zurückgegeben. Es gelten die Bedienreihenfolge nach Ankunft und Verwaltungskosten null.
A läuft von eins bis drei. Sein aktiver Abschnitt beginnt bei eins, verbraucht zwei Einheiten und erzeugt deshalb eine gebündelte Rückgabe $(7, 2)$. Das Budget ist danach null. B kommt bei fünf an und muss trotz freiem Kern bis sieben warten. Bei sieben werden zwei Einheiten verfügbar; B rechnet sieben bis acht eine Einheit. Direkt nach seinem Ende ist das Restbudget eins, und die Rückgabeliste enthält $(13, 1)$. Die Antwortzeiten lauten für A zwei und für B acht minus fünf gleich drei Millisekunden. Eine Rückgabe bei neun wäre falsch: Verwendet wird hier der Abschnittsbeginn eins plus Zeitraum sechs, nicht das Ende drei plus sechs. Ebenso ist eine pauschale Auffüllung bei sechs nicht die Regel des Sporadic Servers.

Folie 43 / 50

Papier 3: ein neues Polling-Taskset nachweisen

15 Minuten · RMS · ein Kern · Kosten und Sperrwartezeit null.

Aufgabe	Bedarf	Periode = Frist
Regler	1	5
Polling Server	2	10
Hintergrundtask	2	20

Auslastung und drei Antwortzeiten berechnen. Darf der Server danach ohne neue Analyse zum Deferrable Server werden?

Notizen 43 / 50

RMS heißt Rate Monotonic Scheduling, feste Priorität nach kürzerer Periode. Die Einheiten sind Millisekunden. Alle Aufgaben sind vollständig präemptiv, also zugunsten höher priorisierter Arbeit unterbrechbar, und besitzen relative Fristen gleich ihrer Periode. Der Polling Server wird mit vollem periodischem Bedarf zwei angenommen. Die Hintergrundtask ist hier eine eigene periodische Aufgabe mit niedrigstem Rang, kein zusätzlicher unbudgetierter Auftrag innerhalb der Serverqueue.
Die Auslastung ist $1/5+2/10+2/20 = 1/2$, also 50 Prozent. Für den Regler ergibt sich Antwortzeit eins. Beim Server startet die Response-Time Analysis, Antwortzeitanalyse, mit zwei und ergibt $2+\lceil 2/5 \rceil \cdot 1 = 3$; drei bleibt unverändert. Für die Hintergrundtask starten wir bei zwei: $2 + \lceil 2/5 \rceil \cdot 1 + \lceil 2/10 \rceil \cdot 2 = 5$. Mit fünf bleibt derselbe Wert, weil $\lceil 5/5 \rceil = 1$. Die drei Antwortzeiten sind eins, drei und fünf, alle rechtzeitig. Der Austausch gegen einen Deferrable Server ist damit nicht nachgewiesen. Dessen spät verbrauchtes altes Budget und neues Budget können kurz hintereinander stören; die periodische Interferenzannahme der Rechnung muss neu geprüft werden.

Folie 44 / 50

Papier 4: Frist und Überlast begründet beurteilen

10 Minuten · zwei kurze Entscheidungen mit Rechnung.

1. Papier 2: B kommt bei 5, relative Frist 2 ms. Ist sein Abschluss rechtzeitig? Ändert $T = 6$ seine Frist?
2. Server mit $Q = 1, T = 4$: dauerhaft 2 ms neue Arbeit je 4 ms. Reicht eine größere Queue für dauerhafte Rechtzeitigkeit?

Nennen Sie jeweils die fehlende oder verletzte Zusicherung.

Notizen 44 / 50

Im ersten Fall bezieht sich B auf den Auftrag aus Papier 2: Freigabe fünf, eigener Bedarf eins und Abschluss acht im klassischen Sporadic-Server-Spezialfall. Die relative Frist von zwei Millisekunden ergibt die absolute Deadline fünf plus zwei gleich sieben. Acht liegt eine Millisekunde zu spät. Der Serverzeitraum $T = 6$ bestimmt Wiederauffüllungen und ändert diese fachliche Auftragsfrist nicht. Eine korrekte Budgetbuchhaltung kann also mit einer Auftragsfristverletzung zusammenfallen. Für eine Zusage wäre eine geeignete Last- und Antwortzeitanalyse oder eine andere Bedienkonfiguration erforderlich.
Im zweiten Fall ist Q die Kapazität und T der Zeitraum. Die nominelle Bedienkapazität ist ein Viertel des Prozessors, der dauerhaft geforderte Anteil zwei Viertel. Selbst bei vollständiger möglicher Bedienung bleibt im Mittel eine Millisekunde Arbeit pro Eingangsintervall zusätzlich unerledigt. Eine größere Queue, also Warteschlange, verschiebt höchstens das Überlaufen. Sie erhöht die Rechengeschwindigkeit nicht und begrenzt keine dauerhafte Wartezeit. Erforderlich sind beispielsweise weniger angebotene Arbeit, mehr nachgewiesene Bedienkapazität oder eine fachlich zulässige Annahme- und Verwerfungsregel. Eine endliche Queue ist eine Speichergrenze und kein Fristnachweis.

Folie 45 / 50

Praxis: Polling als sichtbares Modell ausführen

Übungsblatt: Aufgabe 6 · Programmierleitfaden

- Cortex-M4 mit FreeRTOS, zwei Threads
- Modellthread: virtuelle Zeit und Polling-Zustand
- Heartbeatthread: unabhängiges Lebenszeichen
- LED-Anzeige macht Modellentscheidungen sichtbar.

Die Simulation ist kein realer Kernelserver.

Notizen 45 / 50

Cortex-M4 ist der verwendete Prozessorkern des STM32H755. FreeRTOS ist das Echtzeitbetriebssystem der bekannten Basis aus Vorlesung 3. Ein Thread ist ein vom Kernel verwalteter Ausführungspfad. Im selbst implementierten Versuch gibt es einen Modellthread und einen Heartbeatthread. Heartbeat bedeutet regelmäßiges Lebenszeichen. LED steht für Light Emitting Diode, Leuchtdiode. Eine Leuchtdiode zeigt die virtuelle Bedienentscheidung, die andere das unabhängige Lebenszeichen an.
Aufgabe 6 des verlinkten Übungsblatts enthält den Typ, die Implementierungsregeln und die zu bearbeitenden Funktionen. Der Programmierleitfaden beschreibt die Projektvorbereitung. Verwenden Sie diese Basis und schreiben Sie den Modellschritt sowie die Threadschleifen selbst; vollständiger Lösungscode steht ausschließlich im Lösungsprofil. Der Modellthread simuliert einen Polling Server mit bekannten Arbeitseinheiten. Er misst und begrenzt nicht die tatsächliche eigene Prozessorzeit beliebiger Nutzfunktionen. Das Ziel ist, vor der Ausführung die Modellzustände auf Papier vorherzusagen und danach die angezeigte Folge zu prüfen. Ein tatsächlich ausgeführter Boardlauf ist gesondert zu dokumentieren; die vorliegenden Modellrechnungen behaupten keinen solchen Lauf.

Folie 46 / 50

Praxis: pro Schritt genau eine Modellentscheidung

Virtuelle Aufträge: je eine Einheit bei 1, 3, 7 und 8.

- Modellfelder: `t`, `pending`, `budget`, `served`
- Polling bei `t % 6 == 0`, Kapazität 2
- Erst Ankunft, dann Polling, dann höchstens eine Einheit
- Leere Arbeit: Restbudget verfällt.
- Vor dem nächsten Zyklus Modell zurücksetzen.

Notizen 46 / 50

Die vier Felder gehören zur C-Struktur Modell im Übungsblatt. C ist die verwendete Programmiersprache. t ist die virtuelle Zeit vor einem Schritt, pending die Zahl offener Einheitsaufträge, budget die verfügbare Modellkapazität und served die Zahl bereits bedienter Einheiten. Das Prozentzeichen im C-Ausdruck bedeutet Rest einer ganzzahligen Division. t modulo sechs gleich null kennzeichnet die Pollingzeitpunkte

Notizen, Fortsetzung 46 / 50

null, sechs und zwölf im endlichen Zyklus.
An den virtuellen Zeiten eins, drei, sieben und acht wird zuerst je eine Einheit offener Arbeit hinzugefügt. Bei einem Pollingzeitpunkt wird das Budget auf zwei gesetzt, falls Arbeit wartet, sonst auf null. Danach wird bei positiver offener Arbeit und positivem Budget genau eine Einheit ausgeführt: offene Arbeit und Budget sinken um eins, die Bedienzahl steigt um eins. Wird die Arbeit leer, verfällt Restbudget. Die virtuelle Zeit wird anschließend erhöht. Nach dem bearbeiteten Schritt siebzehn steht die virtuelle Zeit auf achtzehn. Dieser Endzustand kann vor dem Zurücksetzen beobachtet werden; erst vor dem nächsten Zyklus wird der neue Nullzustand angelegt. Alle Aufträge haben genau eine bekannte Modelleinheit, deshalb genügt hier ein Zähler statt einer Nutzdatenqueue. Die Simulation erzeugt daraus keine Laufzeitschranke für beliebige Nachrichtenfunktionen.

Folie 47 / 50

Praxis: erst vorhersagen, dann die Anzeige prüfen

Vor dem Ausführen schriftlich begründen:

- In welchen virtuellen Schritten wird Arbeit bedient?
- Wie lauten alle vier Modellfelder nach Schritt 17?
- Warum kann ein Schritt trotz offener Arbeit keine Einheit bedienen?
- Welche Zeitgröße zeigt die verlangsamte LED-Anzeige?

Anzeigeabstand: `osDelay(100)` Kernel-Ticks, keine CPU-Budgetmessung.

Notizen 47 / 50

Lassen Sie die Studierenden ihre Prognose vor dem Bauen und Flashen festhalten. Die folgenden Ergebnisse dienen erst danach zur Auswertung. Im Modell ist die Queue beim Pollingzeitpunkt null leer. Die bei eins und drei eingehenden Einheiten warten bis sechs. Bei sechs wird die erste, bei sieben die zweite Einheit bedient; die Ankunft bei sieben ist vorher bereits zum Arbeitszähler addiert worden. Nach sieben ist das Budget null. Die Ankunft bei acht erhöht die offene Arbeit nochmals. Bei zwölf und dreizehn werden diese beiden verbleibenden Einheiten bedient. served bedeutet Zahl der bedienten Einheiten, pending Zahl der noch offenen Einheiten. Nach Schritt siebzehn stehen sie auf vier und null; das Budget ist ebenfalls null und die virtuelle Zeit t beträgt achtzehn.
osDelay ist die Wartefunktion der CMSIS-RTOS2-Schnittstelle. CMSIS bedeutet Common Microcontroller Software Interface Standard, RTOS Real-Time Operating System, Echtzeitbetriebssystem. Ein Kernel-Tick ist eine Einheit der Betriebssystemzeitbasis. Bei 1000 Ticks pro Sekunde entsprechen 100 Ticks nominell 100 Millisekunden; die tatsächliche Konfiguration muss geprüft werden. Der Abstand enthält außerdem Modellrechnung und mögliche Verdrängung. CPU heißt Central Processing Unit, Prozessor. Diese Anzeigeverlangsamung ist weder ein 100-Millisekunden-CPU-Budget noch eine Messung des virtuellen Millisekundenbedarfs. Das zweite Lebenszeichen verwendet unabhängig eine Verzögerung von 500 Ticks.

Folie 48 / 50

Quiz: Regeln anhand eines Ereignisses entscheiden

15 Minuten · zehn Fragen mit gemeinsamer Begründung.

- Leere Queue: Was passiert mit Restbudget?
- Budget null: Was wartet, was darf weiterlaufen?
- Welche Rückgabe gehört zu welchem Verbrauch?
- Welche Frist wurde tatsächlich nachgewiesen?

Fragensatz: quiz_06_server-verfahren.csv

Notizen 48 / 50

Die Queue ist die Warteschlange noch unvollständiger Aufträge. Restbudget ist die noch erlaubte eigene Prozessorzeit. Das Quiz prüft, ob diese beiden Zustände getrennt werden und ob die jeweilige Serverregel korrekt angewendet wird. Beim Polling Server verfällt Restbudget nach Leerwerden; beim Deferrable und Sporadic Server bleibt ungenutzte Kapazität gemäß ihren Regeln verfügbar. Bei Budget null müssen unerledigte Serveraufträge auf passende Kapazität warten, während andere bereite Aufgaben weiterhin auf dem Kern laufen können.
Der verlinkte Fragensatz enthält zehn Fragen. Begründen Sie jede Auswahl durch ein Ereignis oder einen Rechenschritt. Beim klassischen Sporadic-Server-Spezialfall gehört die Rückgabe zum aktiven Abschnitt und seinem tatsächlichen Verbrauch, nicht pauschal zur nächsten Periodengrenze. Der nachgewiesene Abschluss einer vollen Polling-Gelegenheit ist wiederum keine pauschale Frist für alle wartenden Aufträge. Wenn eine Antwort ohne Angabe der Regel nur auf den Kapazitäts-Prozentwert verweist, fehlt meist die entscheidende zeitliche Information. Nutzen Sie die gemeinsame Auswertung, um genau diese Verwechslungen zu korrigieren.

Folie 49 / 50

Klausurcheck: sechs Angaben vor jeder Zeitachse

1. Ein Kern, Prioritäten und Verwaltungskosten
2. Auftragsankünfte, Bedarfe und Bedienreihenfolge
3. Anfangsbudget und eindeutige Auffüllregel
4. Ereignisse an derselben Zeitgrenze
5. Eigene CPU-Zeit und vollständige Antwortzeit
6. Auftragsfristen und zulässige Eingangsbelastung

Zur Einstiegsfrage: Mehr Queueplätze erzeugen keine zusätzliche CPU-Zeit.

Notizen 49 / 50

Eine gute Klausurlösung beginnt mit dem Modell. Anzahl der Kerne und feste Prioritäten legen die Konkurrenz fest. Ankünfte und eigene Bedarfe bestimmen die verfügbare Arbeit, die Bedienreihenfolge bestimmt den nächsten Auftrag. Anfangsbudget und Auffüllregel müssen zum benannten Verfahren

Notizen, Fortsetzung 49 / 50

passen. Bei gleichzeitigen Ereignissen werden in unseren Aufgaben fällige Rückgaben und neue Ankünfte vor der nächsten Auswahl berücksichtigt. CPU bedeutet Central Processing Unit, Prozessor; eigene CPU-Zeit ist von der gesamten Antwortzeit ab Freigabe zu unterscheiden.
Markieren Sie anschließend jeden Laufabschnitt und seine verbrauchte Kapazität. Beim Sporadic-Server-Spezialfall führen Sie die Rückgabeliste parallel. Berechnen Sie die Antwortzeit als Abschluss minus Freigabe und vergleichen Sie sie mit der jeweiligen relativen Auftragsfrist, sofern diese gegeben ist. Für einen allgemeinen Nachweis reichen einzelne günstige Läufe nicht. Nennen Sie daher am Ende präzise, ob Sie einen konkreten Ablauf berechnet, eine volle periodische Bediengelegenheit nachgewiesen oder eine Überlastannahme widerlegt haben. Zur Einstiegsfrage lässt sich jetzt präzise antworten: Ein größerer Puffer schützt den Regler nicht vor unbegrenzt ausführungsbereiter Diagnosearbeit. Ein geeignet begrenztes Serverbudget kann deren Einfluss kontrollieren, wenn auch die Serverregel und der Fristnachweis der konkurrierenden Tasks passen. Diese Vorgehensweise bereitet die zeitlich begründete Architekturentscheidung in der integrierten Fallstudie vor.

Folie 50 / 50

Literatur und Anschluss an Multicore

- Giorgio C. Buttazzo: *Hard Real-Time Computing Systems*, 3. Auflage
- B. Sprunt, L. Sha, J. Lehoczky: *Aperiodic Task Scheduling for Hard-Real-Time Systems*, 1989
- FreeRTOS-Dokumentation und CMSIS-RTOS2-API-Referenz

Vorlesung 7: vom lokalen Kernel zur Mehrkernarchitektur.

Notizen 50 / 50

Giorgio C. Buttazzos Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications ist die zentrale Kursliteratur. Die dritte Auflage dient zur fachlichen Vertiefung der Einplanung aperiodischer Arbeit. Der klassische Primärbeitrag von Brinkley Sprunt, Lui Sha und John Lehoczky, Aperiodic Task Scheduling for Hard-Real-Time Systems, erschien 1989 in Real-Time Systems und führt den Sporadic Server ein. Für die Vertiefung ist entscheidend, die jeweilige Servervariante mit ihrer Aktivierungs- und Wiederauffüllregel zu lesen; ähnliche Namen allein garantieren keine gleichen Eigenschaften.
FreeRTOS ist das verwendete Echtzeitbetriebssystem. CMSIS bedeutet Common Microcontroller Software Interface Standard und RTOS Real-Time Operating System; API bedeutet Application Programming Interface, Programmierschnittstelle. Die zugehörigen Referenzen beschreiben Prioritäten, Wartefunktionen und Queueoperationen der konkret eingesetzten Umgebung. Sie ersetzen keine Spezifikation eines zusätzlichen Serververfahrens.
Vorlesung 7 erweitert nun das Ein-Kern-Modell. Multicore bedeutet mehrere Prozessorkerne; eine Firmware ist das für den Mikrocontroller übersetzte Programm. Zwei getrennte Firmwareprogramme benötigen ausdrückliche Regeln für gemeinsamen Datenbesitz und konkurrierende Hardwarezugriffe. Die heutigen Einprozessor-Schranken gelten für diese zusätzliche Konkurrenz nicht automatisch weiter. Vorlesung 8 untersucht anschließend die Aussagekraft von Messungen. In der Fallstudie aus Vorlesung 9 führen wir Eingangsmodell, Serverregel, Datenvertrag und Prüfstatus zusammen. LED bedeutet Light Emitting Diode, Leuchtdiode. Die heutige LED-Simulation liefert dafür Modellverständnis, aber keinen allgemeinen Laufzeitnachweis.

Notizen, Fortsetzung 50 / 50

Die zusätzlichen Architektur-, Warte- und Rückgabedarstellungen wurden anhand von sieben vom Lehrenden bereitgestellten Vorjahres-SVGs neu ausgearbeitet. Sie verwenden die hier eingeführten Modellwerte und Begriffe. Die Zeichnungen sind Lehrmodelle, keine gemessenen FreeRTOS-Abläufe; die Auswahl und fachlichen Anpassungen dokumentiert `docs/reviews/vorlesung06-abbildungen.md`.